RutaAlerta: libro de aprendizaje del proyecto

Grupo 7 · Ingeniería en Informática · Duoc UC San Andrés · 21 de octubre de 2026

Este libro explica RutaAlerta de punta a punta para que cualquier integrante pueda entenderlo, construirlo y defenderlo. Está escrito para leerse en orden, pero cada capítulo se entiende solo.

Cómo usar este libro

Los capítulos avanzan de lo general a lo técnico: primero el proyecto, luego la arquitectura, la base de datos, el código y la gestión. Al final hay preguntas de defensa, un glosario y una autoevaluación.

Parte	Capítulos	Para qué sirve
El proyecto	1 a 8	Saber qué es, para quién, qué se mide y qué se construye
La solución técnica	9 y 10	Entender cómo se conectan las piezas y cómo se protegen los datos
La base de datos	11 a 14	Dominar los conceptos y el modelo de 56 tablas
El código	15	Ver cómo funciona la API inicial y sus pruebas
La gestión	16 a 19	Entender Scrum, las pruebas, los indicadores y los documentos
La defensa	20 a 22	Practicar respuestas, repasar términos y comprobar lo aprendido
Las pantallas	Anexo A	Ver los mockups de las 17 pantallas de la app y del panel, explicados

Tres reglas atraviesan todo el libro. Primero, se distingue siempre lo diseñado, lo implementado y lo pendiente. Segundo, los porcentajes de los objetivos son metas, no resultados. Tercero, RutaAlerta no promete lo que no puede demostrar.

1. Qué es RutaAlerta en una página

RutaAlerta es una plataforma colaborativa que ordena la información de un robo de vehículo en un expediente único y trazable. Tiene una aplicación móvil para la ciudadanía y un panel web para operadores autorizados de seguridad ciudadana.

El nombre completo es «RutaAlerta: red ciudadana y asistencia al operador para la prevención informativa, gestión de incidentes y búsqueda de vehículos reportados por robo». Es el proyecto Capstone del Grupo 7 de Ingeniería en Informática, Duoc UC sede San Andrés, Concepción.

La idea en cinco frases

1. Una persona reporta el robo desde la app y recibe un folio que confirma que su reporte llegó.
2. Un operador autorizado revisa el caso en su bandeja y decide si pide ayuda ciudadana.
3. Si la pide, publica una solicitud con datos mínimos; otros ciudadanos pueden informar avistamientos.
4. Cada avistamiento llega pendiente y una persona lo acepta, lo rechaza o pide aclaración, con un motivo.
5. La persona afectada consulta el estado, el siguiente paso y un canal de apoyo, y puede dar acceso a un contacto de confianza.

Qué la hace distinta

No es un grupo de WhatsApp: ningún aporte se publica como verdad sin revisión humana. Tampoco es un sistema policial: no identifica personas ni persigue vehículos. Su valor está en reunir antecedentes dispersos, dejar registro de quién decidió qué y mantener informada a la víctima sin exponerla.

Ficha rápida

Dato	Valor
Equipo	Jorge Ceballos (Product Owner), Juan Marinan y Alan Bravo (Scrum Master)
Docentes guías	Juan Pablo Mellado Alarcón y Mabel Alejandra Herrera Pino
Duración del curso	14 semanas, desde el 13 de agosto de 2026 hasta el 18 de noviembre de 2026
Metodología	Scrum con sprints de una semana
Tecnologías	Flutter (móvil), React (panel), FastAPI (API), PostgreSQL con PostGIS (base de datos), Azure (nube)
Tamaño del diseño	6 objetivos, 31 casos de uso, 56 tablas, 549 columnas, 110 claves foráneas
Repositorio	github.com/AlanBravo19/Capstone_2026_G7

2. El problema, el contexto y lo que no prometemos

El problema central es la dispersión: ante un robo, los antecedentes quedan repartidos entre la víctima, los testigos, mensajes, formularios y archivos. Eso impide saber qué se recibió, cuándo ocurrió cada observación y quién la revisó.

Tres personas, tres dolores

- **La persona afectada** necesita registrar lo ocurrido, saber que su información llegó y conocer el siguiente paso.
- **El testigo** puede tener una foto o una observación útil, pero no sabe a quién entregársela de forma segura.
- **El operador** de una central tiene personal y quizás cámaras, pero le cuesta relacionar datos sueltos y saber cuáles están confirmados.

Público

El público es general: propietarios, conductores, pasajeros y testigos. Los taxistas y conductores de aplicaciones son un posible grupo piloto por su frecuencia de desplazamiento, no los únicos usuarios. El piloto académico se ubica en Concepción, con participantes y organizaciones ficticias.

Diferenciación defendible

En Chile ya existen plataformas institucionales con lectura de patentes e inteligencia artificial, como SITIA. RutaAlerta no se presenta como la primera plataforma de IA vehicular. Su hipótesis de valor es integrar la colaboración ciudadana revisada, el apoyo al afectado y recomendaciones explicables al operador.

Lo que RutaAlerta no promete

Estos límites son parte del diseño y conviene saberlos de memoria:

- No predice delitos ni intenciones.
- No garantiza rutas seguras: compara alternativas con los datos disponibles y se abstiene si faltan.
- No garantiza recuperar el vehículo.
- No sigue un vehículo robado con el GPS del teléfono de la víctima: ese GPS ubica a la persona, no al auto.
- No identifica delincuentes ni usa reconocimiento facial.
- No despacha policía ni organiza persecuciones ciudadanas.
- No reemplaza la denuncia oficial.
- No reduce clínicamente el trauma: el apoyo es informativo.
- No tiene convenio con municipalidades ni con Carabineros; cualquier integración real exige un acuerdo formal.

Si te preguntan por qué tantos límites, la respuesta es simple: el proyecto sólo afirma lo que puede medir y demostrar.

3. Objetivos y cómo se miden

Los objetivos de RutaAlerta miden resultados del uso de la plataforma en un piloto, no tareas de programación. Crear una API o diseñar una tabla son actividades; un objetivo dice qué mejora para el usuario, cuánto y cuándo.

Objetivo general

Contribuir a una respuesta ciudadana e institucional más informada, oportuna y trazable frente a incidentes de robo vehicular, mediante una plataforma colaborativa de orientación territorial, gestión de antecedentes, apoyo al afectado y asistencia explicable al operador, evaluada en un piloto al término de las 14 semanas del curso.

Objetivos específicos

ID	Nombre	Meta	Cuándo	Estado
OE1	Decisiones preventivas informadas	Al menos 80 % de 5 a 10 participantes interpreta bien antecedentes y rutas en dos escenarios, sin ayuda	Semana 14	Comprometido
OE2	Coordinación más oportuna	Reducir al menos 20 % la mediana del tiempo para reunir antecedentes, frente a archivos separados; 12 pares de ensayos	Semanas 13 y 14	Comprometido
OE3	Colaboración útil y trazable	Al menos 18 de 20 avistamientos útiles, vinculados al caso y revisados en 5 minutos	Semanas 13 y 14	Comprometido
OE4	Autonomía en el seguimiento	Al menos 80 % completa estado, siguiente paso, apoyo y contacto de confianza en 3 minutos, sin ayuda	Semana 14	Comprometido
OE5	Búsqueda de evidencia eficiente	Reducir al menos 20 % la mediana del tiempo hasta una observación correcta, con asistente frente a búsqueda manual	Semanas 13 y 14	Propuesto, sin ratificar
OE6	Interpretación de indicadores	Al menos 10 de 12 tareas de análisis correctas, cada una en 3 minutos o menos	Semana 14	Propuesto, sin ratificar

OE1 a OE4 vienen de la propuesta original. OE5 y OE6 son la ampliación de la versión 4.1 del libro y requieren que los docentes los ratifiquen.

Conceptos para entender las metas

- **Mediana:** el valor del medio cuando ordenas los tiempos de menor a mayor. Se usa en vez del promedio porque un caso muy lento no la distorsiona.
- **Ensayo emparejado:** la misma tarea se hace dos veces, una con el método antiguo y otra con la plataforma, alternando el orden para no favorecer a ninguno.
- **Reducción porcentual:** se calcula así:

```
\text{reducción} = \frac{\text{mediana de referencia} - \text{mediana con plataforma}}
{\text{mediana de referencia}}
```

- **Fracaso con límite:** en OE2 y OE5, un intento sin respuesta correcta a los 10 minutos se registra como fracaso y cuenta con ese límite; no se borran intentos difíciles.
- **Muestra exploratoria:** con 5 a 10 personas no se puede generalizar a todo Chile; los resultados muestran factibilidad, no eficacia nacional.

Objetivo, requisito y caso de uso no son lo mismo

El objetivo OE2 dice «reducir 20 % el tiempo». El caso de uso CU13 «consultar casos y alertas» es una acción que ayuda a lograrlo. La tabla `caso` es una pieza de implementación. Ninguna de esas piezas demuestra por sí sola que OE2 se cumplió: sólo la medición en el piloto lo demuestra.

4. Alcance y estado real

El alcance se divide en cuatro niveles, y sólo el primero está comprometido para el semestre. Al 14 de octubre de 2026 el diseño está completo, la base de datos y una API inicial están probadas, y la app y el panel aún no tienen código.

Los cuatro niveles del alcance

Nivel	Qué incluye	Situación
Núcleo del MVP	Cuenta, vínculo vehicular revisable, reporte con folio, evidencia, colaboración ciudadana revisada, revisión institucional, seguimiento y contacto de confianza	Comprometido: 88 puntos del backlog
Orientación y analítica	Antecedentes territoriales, comparación de rutas, avisos de trayecto, indicadores M01 a M18	A acordar con los docentes; se propone alcance reducido
Ampliación piloto	Cámaras autorizadas, recorridos posibles, comparación visual, copiloto con IA	Diseñada; fuera de esta entrega mientras no se ratifique
Evolución posterior	Integración municipal o policial, muchos proveedores de cámaras	Fuera del curso; requiere convenios inexistentes

MVP significa producto mínimo viable: la versión más pequeña que permite probar el recorrido completo con usuarios. En RutaAlerta, el criterio de cierre es demostrar en vivo: reporte, recepción, revisión, publicación, avistamiento, decisión, notificación y seguimiento.

Qué existe hoy

Componente	Estado	Dónde verlo
Libro del proyecto v4.1: alcance, objetivos, requisitos y arquitectura	Completado (diseño)	Evidencias de documentación

Componente	Estado	Dónde verlo
31 casos de uso y diagramas UML	Completado (diseño)	Base de datos, Diagramas_UML.pdf
Modelo de datos: 56 tablas, 549 columnas, 110 claves foráneas	Implementado y verificado: 36 de 36 pruebas	Base de datos, carpeta sql
API FastAPI inicial: registro, sesión, reporte con folio y bandeja	Implementada y probada: 10 de 10 pruebas	Aplicación, carpeta backend
App móvil Flutter	Pendiente, sin código	Sprints 1 a 3
Panel web React	Pendiente, sin código	Sprints 1 a 3
Despliegue en Azure	Pendiente	Sprint 3
Cámaras, recorridos e IA	Diseñados en tablas; sin implementar ni convenio	Tablas del módulo 06 y 07

Por qué hay retraso

El plan del libro esperaba un primer flujo desplegado en la semana 8 y el panel en la semana 9. El tema del proyecto se definió en la semana 6 sin reiniciar el plazo de 14 semanas, y las semanas 7 a 9 se usaron para rediseñar alcance, objetivos y modelo de datos.

La respuesta fue replanificar en cinco sprints semanales y priorizar el MVP. Esa reducción de alcance se propone a los docentes; no se considera aprobada hasta que ellos lo digan.

5. Actores, roles y permisos

Un actor es quien interactúa con el sistema desde afuera: una persona con un rol o un sistema externo. RutaAlerta tiene seis actores primarios y cuatro sistemas externos, y cada uno ve sólo lo que necesita.

Actores primarios

Actor	Qué puede hacer	Qué no puede hacer
Ciudadano	Crear su cuenta, registrar vehículos, reportar, ver solicitudes públicas y aportar avistamientos	Ver expedientes ajenos; no necesita vehículo propio para colaborar
Persona afectada	Es el ciudadano que reportó el caso: consulta estado, siguiente paso y apoyo, y gestiona su contacto de confianza	Ver la coordenada precisa ni cámaras
Contacto de confianza	Consultar el seguimiento limitado de un caso mientras tenga acceso vigente	Ver la ubicación precisa; pierde el acceso al ser revocado
Operador	Ver casos y cámaras de su organización, revisar aportes, registrar gestiones	Ver casos de otras organizaciones

Actor	Qué puede hacer	Qué no puede hacer
Supervisor	Todo lo del operador, más asignar responsables	Asignar a miembros de otra organización
Administrador	Gestionar usuarios, organizaciones, fuentes, recursos de apoyo y cámaras	Ver expedientes privados por defecto
Evaluador	Ejecutar escenarios de prueba y registrar mediciones del piloto	Operar casos reales

Sistemas externos

- **Proveedor de mapas y rutas** (por ejemplo, Valhalla): calcula recorridos, no decide qué ruta es segura.
- **Proveedor de notificaciones**: transporta avisos; que el proveedor acepte un envío no significa que la persona lo leyó.
- **Almacenamiento privado de archivos** (Azure Blob Storage): guarda fotos y videos; la base sólo guarda sus datos y su huella.
- **Sistema de video autorizado**: entrega transmisión o grabación mediante un conector; no hay convenio aún.

Flutter, FastAPI, PostgreSQL y el motor de IA no son actores: son componentes internos del sistema.

Tres ideas clave sobre permisos

1. **El registro público sólo crea ciudadanos.** Los roles de operador, supervisor y administrador se conceden por un flujo administrativo, nunca desde la app.
2. **La organización define el ámbito.** Un operador pertenece a una organización mediante una membresía, y sólo ve los casos de esa organización.
3. **Conocer una patente no da acceso.** Un usuario se relaciona con un vehículo mediante un vínculo revisable; saber la patente no lo convierte en dueño.

Supervisor como especialización

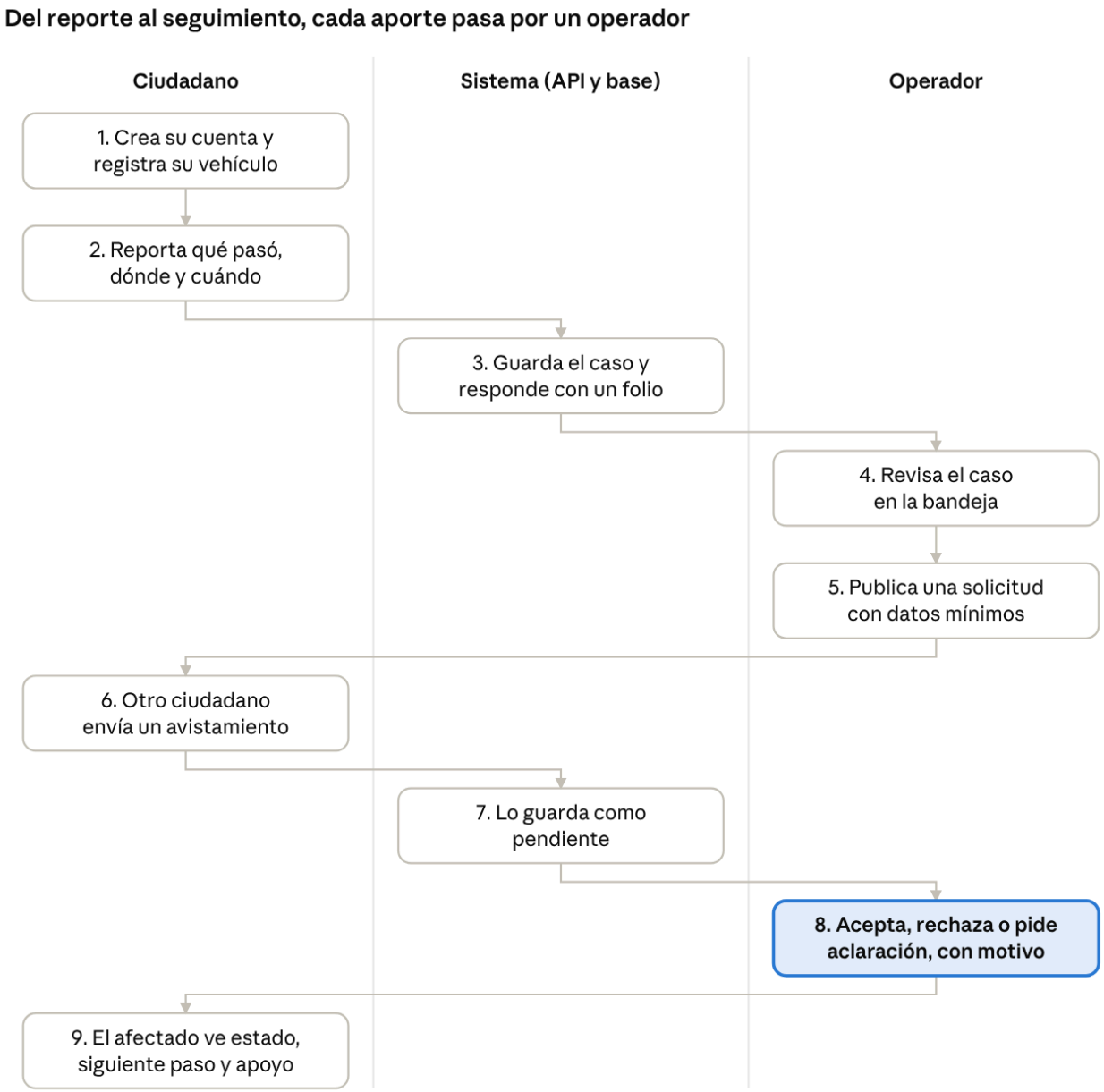
En UML, el supervisor *generaliza* al operador: hereda todos sus casos de uso y agrega uno, asignar casos. En el diagrama se dibuja con una flecha de triángulo vacío que apunta al operador.

6. Cómo funciona: recorridos y estados

El recorrido central va del reporte al seguimiento y pasa siempre por una revisión humana. Ningún aporte ciudadano entra al caso como verdad sin que un operador decida, con motivo.

El recorrido completo

El dibujo de abajo muestra quién hace cada paso: ciudadano, sistema u operador.



recorrido de un caso · 9 pasos en 3 carriles

Si la red falla en el paso 3, la app reintenta con la misma clave y recibe el mismo folio. Un avistamiento aceptado en el paso 8 es un antecedente revisado, no una recuperación.

Cada paso tiene su pantalla dibujada en el **Anexo A: mockups de todas las pantallas**, al final del libro.

Antes del incidente

La persona crea su cuenta y registra su vehículo con marca, modelo, patente y fotos. El vínculo queda pendiente de revisión, porque conocer una patente no acredita propiedad. También puede consultar antecedentes territoriales con fuente y fecha, y activar avisos durante un trayecto.

El reporte

1. Desde un lugar seguro, la persona indica qué pasó, dónde y cuándo.
2. La app propone la ubicación del GPS y permite corregirla; se guardan la precisión en metros y la hora de captura.
3. La app envía el reporte con una clave única de envío (clave idempotente).
4. El servidor guarda el caso y responde con un folio, por ejemplo `RA-20261014-4F2A1C`.
5. Si la red falla y la app reintenta con la misma clave, el servidor devuelve el mismo folio: no se crea un caso duplicado.

La app distingue cuatro situaciones: borrador local, envío pendiente, recepción confirmada y carga fallida. Guardar un borrador nunca se presenta como atención institucional.

La colaboración ciudadana

1. El operador revisa el caso y decide si pide ayuda.
2. Si la pide, publica una solicitud con datos mínimos: vehículo, sector aproximado y fecha de vencimiento.
3. Otro ciudadano ve la solicitud en el catálogo y envía un avistamiento con hora, lugar y observación.
4. El avistamiento llega **pendiente**. El operador lo acepta, lo rechaza o pide aclaración, siempre con un motivo.
5. Esa decisión queda en un historial que no se puede borrar ni modificar.

El seguimiento del afectado

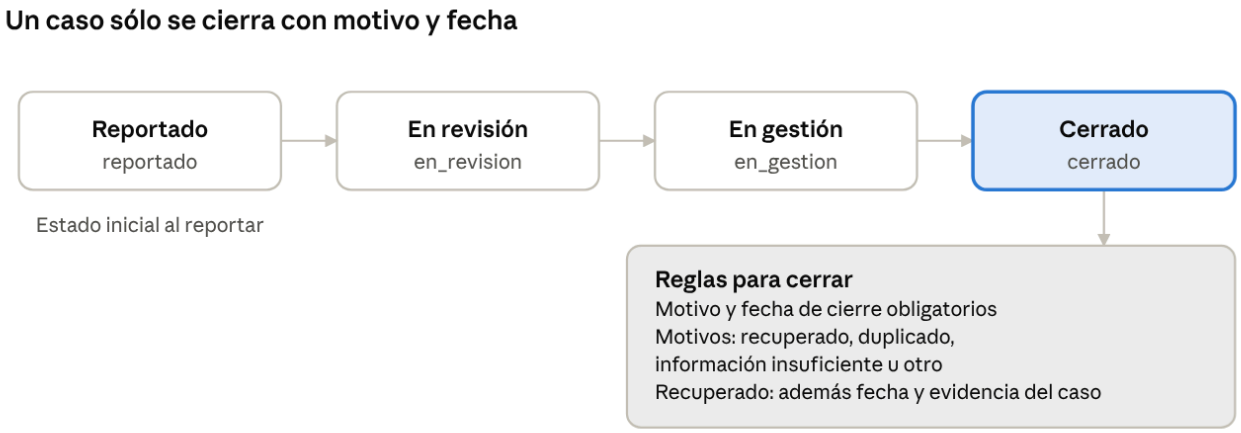
La persona afectada ve el estado, las gestiones comunicables, el siguiente paso y un canal de apoyo verificado. Por defecto ve sólo un sector aproximado y la hora de la observación; la coordenada precisa queda en el ámbito institucional. Puede dar y quitar acceso a un contacto de confianza.

La operación institucional

La bandeja ordena los casos por antigüedad, estado y prioridad, sólo de la organización del operador. Urgencia y veracidad se muestran por separado: un caso puede ser urgente y aún no estar verificado. El supervisor asigna un responsable; sólo puede haber una asignación vigente por caso.

Los estados del caso

Un caso pasa por cuatro estados. Cerrar exige un motivo y una fecha, y cerrar como recuperado exige además una evidencia del mismo caso.



estados del caso · 4 estados y reglas de cierre

La regla también funciona al revés: un caso abierto no puede tener motivo ni fecha de cierre. La restricción `ck_caso_20` lo impide en la base.

Estados de otros objetos

Objeto	Estados posibles	Regla clave
Avistamiento	Pendiente, aceptado, rechazado, requiere aclaración	Toda revisión guarda autor, hora y motivo
Solicitud de colaboración	Publicada, vencida, retirada	Vencida o retirada no acepta nuevos aportes
Evidencia	Pendiente, en carga, recibida, fallida	Recibida no significa auténtica
Vínculo de vehículo	Pendiente, autorizado, rechazado, revocado	Rechazado o revocado exige fecha de fin
Recomendación de IA	Generada, revisada, aceptada, descartada, indeterminada	El modelo nunca decide solo

Un avistamiento aceptado es un antecedente revisado, no una recuperación confirmada. Esa distinción aparece en varias preguntas de defensa.

Cuando faltan recursos

Sin conexión, la app guarda la operación pendiente y avisa el límite. Sin organización receptora, el caso queda sin asignar y se muestran canales verificados. Una persona conduciendo no debe manipular formularios para cumplir el flujo.

7. Requisitos, casos de uso, historias y trazabilidad

En RutaAlerta el mismo número une tres piezas: el requisito RF05, el caso de uso CU05 y la historia HU05 hablan de lo mismo. Esa regla permite seguir cualquier función desde el objetivo hasta su prueba.

Los cuatro conceptos

- **Requisito funcional (RF):** una obligación verificable del sistema. Ejemplo RF03: obtener un folio persistente y que un reintento con la misma clave no duplique el caso.
- **Caso de uso (CU):** una acción que un actor realiza con el sistema para lograr algo. Ejemplo CU03: «Reportar incidente», actor ciudadano.
- **Historia de usuario (HU):** el mismo requisito escrito desde el usuario, para planificar el trabajo. Formato: «Como [rol], necesito [funcionalidad], con la finalidad de [razón]».
- **Criterio de aceptación:** una condición comprobable que dice cuándo la historia está terminada. Se escribe como contexto, evento y resultado esperado.

Ejemplo completo: HU03 Reportar incidente

Como ciudadano afectado por un robo o intento, **necesito** reportar el incidente indicando qué ocurrió, dónde y cuándo, **con la finalidad de** obtener un folio y un estado que confirmen que mi reporte fue recibido.

Escenario	Contexto	Evento	Resultado esperado
Reporte confirmado	La persona tiene sesión y conexión	Envía hora, lugar y relato	El sistema responde folio, estado y fecha de recepción
Reintento sin duplicado	La red falla después del envío	La app reintenta con la misma clave	El sistema devuelve el mismo folio y no crea otro caso
Sin conexión	No hay red	La persona guarda el reporte	Queda como borrador local, sin afirmar atención institucional
Ubicación corregida	El GPS propone una ubicación imprecisa	La persona la corrige en el mapa	Se guarda la ubicación elegida con su precisión

Relaciones entre casos de uso

- **«include» (incluye):** un caso siempre ejecuta otro. CU09 «Comparar alternativas de ruta» incluye CU08 «Consultar antecedentes territoriales».
- **«extend» (extiende):** un caso agrega un comportamiento opcional a otro. CU04 «Aportar evidencia» extiende CU03 y CU05 cuando se adjunta un archivo; CU25 «Recibir aviso» extiende CU10 cuando se cumple la condición de aviso.
- **Generalización:** un actor hereda de otro. El supervisor hereda los casos del operador.

Los 31 casos de uso

CU	Acción	Actor	Objetivos	Prioridad
CU01	Administrar cuenta	Ciudadano	OE1 a OE6 (habilitante)	Must, Sprint 1
CU02	Registrar vehículo	Ciudadano	OE2, OE3	Must, Sprint 2
CU03	Reportar incidente	Ciudadano	OE2, OE4	Must, Sprint 1
CU04	Aportar evidencia	Ciudadano	OE2, OE3	Must, Sprint 2
CU05	Comunicar avistamiento	Ciudadano	OE3	Must, Sprint 3
CU06	Consultar seguimiento y orientación	Ciudadano, contacto	OE4	Must, Sprint 3
CU07	Autorizar contacto de confianza	Ciudadano	OE4	Must, Sprint 3
CU08	Consultar antecedentes territoriales	Ciudadano	OE1	Should
CU09	Comparar alternativas de ruta	Ciudadano	OE1	Could
CU10	Iniciar trayecto con avisos	Ciudadano	OE1	Could
CU11	Consultar solicitudes ciudadanas	Ciudadano	OE3	Must, Sprint 3
CU12	Configurar avisos y privacidad	Ciudadano	OE1, OE4 (habilitante)	Should
CU13	Consultar casos y alertas	Operador	OE2	Must, Sprint 1
CU14	Revisar aportes y duplicados	Operador	OE2, OE3	Must, Sprint 2
CU15	Asignar caso	Supervisor	OE2	Must, Sprint 4
CU16	Registrar gestión y cierre	Operador	OE2, OE4	Must, Sprint 4
CU17	Publicar solicitud de colaboración	Operador	OE3	Must, Sprint 2
CU18	Consultar indicadores y exportar	Operador	OE2, OE6	Should
CU19	Administrar usuarios y permisos	Administrador	OE2, OE3, OE4 (habilitante)	Should
CU20	Gestionar fuentes históricas	Administrador	OE1 (habilitante)	Should
CU21	Revisar datos territoriales	Administrador	OE1, OE3 (habilitante)	Should
CU22	Mantener recursos de apoyo	Administrador	OE4 (habilitante)	Must, Sprint 4
CU23	Consultar auditoría	Administrador	OE2, OE3, OE4 (habilitante)	Should
CU24	Ejecutar escenarios de evaluación	Evaluador	OE1 a OE6 (habilitante)	Should
CU25	Recibir aviso territorial	Ciudadano	OE1	Could

CU	Acción	Actor	Objetivos	Prioridad
CU26	Administrar cámaras autorizadas	Administrador	OE5	Won't (ampliación)
CU27	Consultar video autorizado	Operador	OE2, OE5	Won't (ampliación)
CU28	Explorar recorridos posibles	Operador	OE5	Won't (ampliación)
CU29	Comparar candidatos visuales	Operador	OE5	Won't (ampliación)
CU30	Consultar copiloto del caso	Operador	OE2, OE5	Won't (ampliación)
CU31	Revisar recomendaciones de búsqueda	Operador	OE5	Won't (ampliación)

Habilitante significa que el caso no mejora un resultado por sí mismo, pero sin él los demás no funcionan; por ejemplo, la cuenta.

Trazabilidad: la cadena completa

Trazabilidad es poder seguir cada pieza hacia atrás y hacia adelante. Ejemplo con OE3:

1. **Objetivo:** OE3, al menos 18 de 20 aportes útiles y revisados en 5 minutos.
2. **Requisitos:** RF05 avistamiento, RF11 solicitudes, RF14 revisión, RF17 publicación.
3. **Casos de uso:** CU05, CU11, CU14 y CU17.
4. **Historias:** HU05, HU11, HU14 y HU17, en los Sprints 2 y 3.
5. **Pruebas:** CP11, CP18, CP20 y CP33, ya ejecutadas en la base de datos; CP52, planificada para el piloto.
6. **Evidencia:** los resultados guardados y los 20 ejercicios de las semanas 13 y 14.

Además de las 31 historias hay dos **habilitadores técnicos**: HT01 (entorno reproducible e integración continua) y HT02 (despliegue del piloto). No son funciones para el usuario, pero sin ellos nada se puede entregar.

8. Requisitos no funcionales (calidad)

Un requisito no funcional (RNF) no dice qué hace el sistema, sino qué tan bien lo hace: seguridad, rapidez, resistencia a fallas, facilidad de uso. Cada RNF de RutaAlerta tiene una métrica para poder probarlo.

ID	Atributo	Qué exige, en simple	Cómo se comprueba
RNF01	Autorización	Todo intento prohibido se rechaza: caso ajeno, otra organización, contacto revocado, archivo o cámara sin permiso	Batería de permisos E06

ID	Atributo	Qué exige, en simple	Cómo se comprueba
RNF02	Actualización	Al menos 95 de 100 eventos visibles en el panel en 5 segundos, con 10 sesiones	Prueba de carga
RNF03	Reconexión	20 cortes de red sin perder operaciones confirmadas ni duplicarlas	Ensayo E05
RNF04	Compatibilidad	Flujos principales en un Android y un iPhone físicos y dos navegadores	Matriz de dispositivos E07
RNF05	Evidencia	Validar tipo, tamaño y huella; archivos privados; el original nunca se sobrescribe	Pruebas de carga de archivos
RNF06	Accesibilidad	Controles legibles y etiquetados; tareas hechas sin ayuda	Heurísticas de Nielsen y tareas con usuarios
RNF07	Auditoría	Toda decisión deja actor, hora y contexto; nadie puede editar ese historial desde la app	Revisión de bitácora
RNF08	Recuperación	Restaurar un respaldo en otro entorno en menos de 2 horas	Ensayo E13
RNF09	Analítica	Los totales de M01 a M18 coinciden con consultas de control; sin datos no se inventa un cero	Ensayo E12
RNF10	IA trazable	Toda salida de IA guarda versión y evidencia usada; si la IA falla, la gestión manual sigue	Ensayo E11
RNF11	Rendimiento IA	Respuesta en 10 segundos para un lote de hasta 100 imágenes, medida en hardware declarado	Medición
RNF12	Operación	Despliegue reproducible, secretos fuera del repositorio, datos de prueba separados	Revisión de despliegue

Ideas que conviene entender

- **Idempotencia (RNF03):** repetir la misma solicitud no crea una segunda operación. Es lo que impide dos casos iguales si la red falla.
- **Percentil 95 (p95):** el 95 % de las respuestas fue igual o más rápido que ese valor. Mide la experiencia de casi todos, no sólo el promedio.
- **Heurísticas de Nielsen:** diez reglas clásicas para revisar si una interfaz es fácil de usar, como «visibilidad del estado del sistema» o «prevención de errores».
- **Secretos fuera del repositorio (RNF12):** contraseñas y llaves nunca se suben a GitHub; se leen de variables de entorno o de un gestor de secretos.

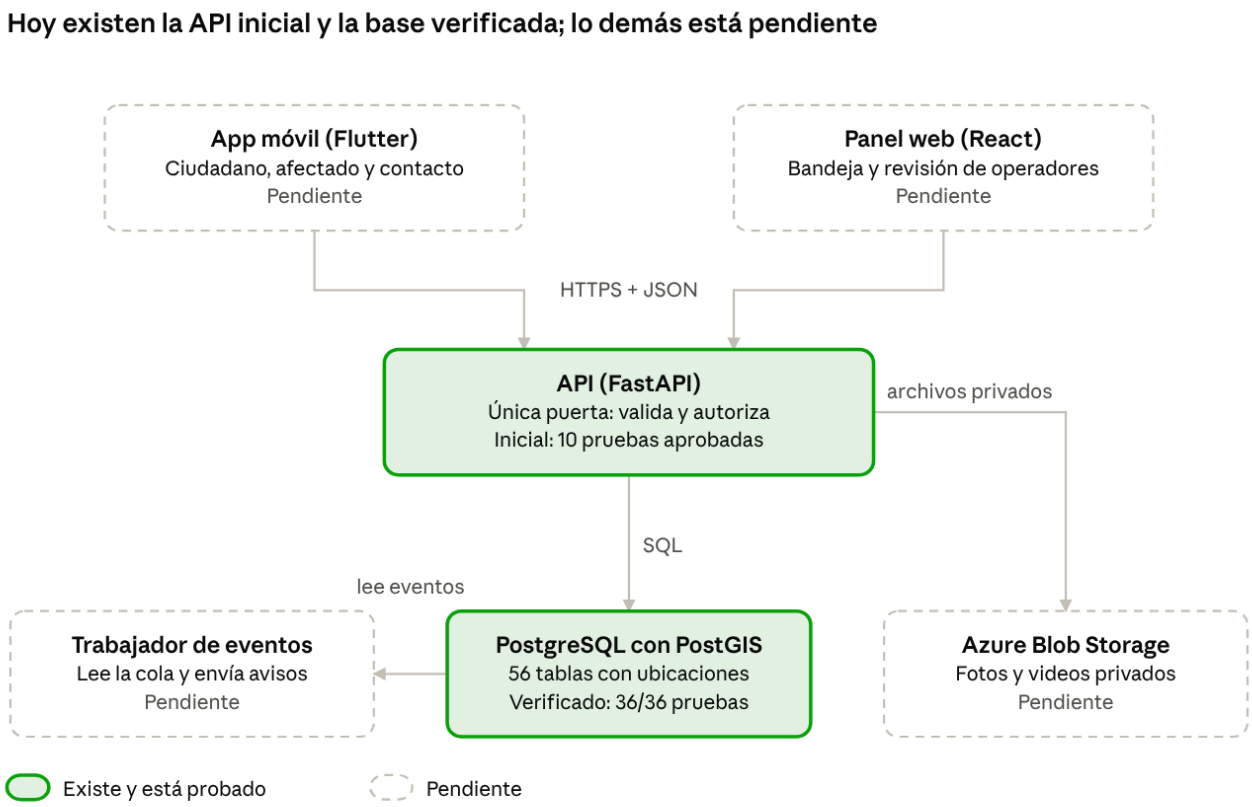
Los umbrales son metas de aceptación. Hay que contrastarlos con mediciones reales; RNF08 no es una garantía comercial de cero pérdida.

9. Arquitectura y tecnologías

RutaAlerta usa una arquitectura cliente-servidor en la que la API es la única puerta a la base de datos. La app móvil y el panel web nunca se conectan directamente a PostgreSQL: siempre piden los datos a la API.

La arquitectura en una imagen

El dibujo de abajo muestra cada pieza, cómo se comunican y su estado real.



arquitectura · 6 piezas y su estado real

La app y el panel nunca se conectan directo a la base: todo pasa por la API. El despliegue del piloto en Azure también está pendiente.

Las piezas y su función

Pieza	Qué hace	Analogía
App móvil (Flutter)	Pantallas para ciudadano, afectado y contacto de confianza	El mesón de atención al público
Panel web (React)	Bandeja, revisión, asignación e indicadores para operadores	La oficina interna de la central

Pieza	Qué hace	Analogía
API (FastAPI)	Recibe pedidos, valida permisos, aplica reglas y habla con la base	El guardia que revisa credenciales y lleva los papeles
Trabajador de eventos	Lee eventos pendientes y envía avisos sin frenar la API	El cartero que reparte después
PostgreSQL con PostGIS	Guarda datos y ubicaciones con reglas de integridad	El archivo central con mapas
Azure Blob Storage	Guarda fotos y videos privados	La bóveda de evidencia

Tecnologías explicadas

Tecnología	Qué es	Para qué se usa en RutaAlerta
Flutter (lenguaje Dart)	Kit de Google para hacer apps Android e iOS con un solo código	La app ciudadana
React	Biblioteca de JavaScript para construir interfaces web	El panel de operadores
FastAPI (Python)	Framework para crear APIs rápidas, con documentación automática	El servidor de reglas y datos
Pydantic	Biblioteca que valida datos de entrada y salida	Rechazar formularios mal llenados
Uvicorn	Servidor que ejecuta la API	Levantar FastAPI
OpenAPI	Estándar para describir una API: rutas, datos y errores	La página `/docs` con el contrato
PostgreSQL	Sistema de base de datos relacional de código abierto	Guardar todo con integridad
PostGIS	Extensión de PostgreSQL para datos geográficos	Distancias, zonas y recorridos
Azure	Nube de Microsoft	Desplegar el piloto: Container Apps, base administrada, Blob Storage, Key Vault
Docker y Docker Compose	Empaquetan cada servicio en un contenedor que corre igual en cualquier equipo	Levantar el entorno con un comando
Git y GitHub	Control de versiones y repositorio en línea	Historial de cambios y aporte de cada integrante
GitHub Actions	Automatización en GitHub	Correr pruebas en cada cambio (integración continua)
pytest	Herramienta de pruebas en Python	Las 10 pruebas de la API

Tecnología	Qué es	Para qué se usa en RutaAlerta
PGlite	PostgreSQL compilado para correr dentro de Node.js	Verificar el esquema sin instalar PostgreSQL
JSON	Formato de texto para intercambiar datos	El idioma entre la app y la API
HTTPS	Protocolo web cifrado	Que nadie lea los datos en el camino

Decisiones y su porqué

- **Monolito modular en la API.** Una sola aplicación dividida en módulos claros (rutas, servicios, repositorios). Con 3 personas y 5 semanas, es más simple de desplegar y depurar que muchos microservicios.
- **PostgreSQL con PostGIS en vez de MySQL.** El problema es territorial: cercanía de avistamientos, cobertura de cámaras y recorridos. PostGIS resuelve esas consultas en la misma base. La decisión se tomó en el libro versión 4.1.
- **Cola transaccional (outbox).** Cuando se guarda un caso, en la misma transacción se guarda un evento en `evento_salida`. Después, el trabajador lo publica. Así nunca se avisa de algo que no quedó guardado.
- **Clave idempotente.** Cada reporte lleva una clave única; un reintento devuelve el mismo folio.
- **IA como servicio separado.** Si la IA falla o es lenta, el registro de incidentes sigue funcionando.
- **Archivos fuera de la base.** La base guarda sólo la ubicación privada, el tipo, el tamaño y la huella del archivo.

Patrones de diseño usados

Patrón	Qué significa	Dónde se aplica
Repositorio	Un módulo concentra las consultas a la base	Consultas SQL de la API
Capa de servicio	Las reglas de negocio viven aparte de las rutas	Estados del caso, revisión
Outbox transaccional	El evento se guarda junto con el cambio	Tabla `evento_salida`
Inyección de dependencias	FastAPI entrega a cada ruta la conexión y el usuario	`Depends(...)` en el código
DTO o esquema	Objetos que definen qué entra y sale de la API	Modelos Pydantic
Registro de solo anexado	Un historial al que sólo se agregan filas	Triggers sobre revisiones y auditoría

El modelo 4+1

El documento de arquitectura (DAS) describe el sistema con cinco vistas, porque cada interesado necesita ver algo distinto:

1. **Vista de escenarios (el +1):** casos de uso; une a las demás.
2. **Vista lógica:** clases y modelo de datos; qué información existe.

3. **Vista de desarrollo:** componentes y carpetas del código.
4. **Vista de procesos:** qué pasa en el tiempo, con diagramas de actividad y de secuencia.
5. **Vista física:** en qué máquinas o contenedores corre cada pieza.

10. Seguridad y privacidad

RutaAlerta maneja relatos, ubicaciones y fotos sensibles, así que la seguridad se construye en capas: la API autoriza cada acceso y la base impide combinaciones inválidas aunque la API fallara.

Autenticación y autorización no son lo mismo

- **Autenticación:** comprobar quién eres. En RutaAlerta, con correo y contraseña, que entregan un token de sesión.
- **Autorización:** decidir qué puedes hacer. Un operador autenticado igual no puede ver casos de otra organización.

Cómo se guardan contraseñas y tokens

- **Contraseñas:** nunca se guardan tal cual. Se guarda un *hash* con el algoritmo scrypt y una sal aleatoria. Un hash es una huella irreversible: sirve para comprobar, no para recuperar la contraseña.
- **Tokens de sesión:** el usuario recibe un token aleatorio; la base guarda sólo su huella SHA-256 de 32 bytes. Si alguien robara la tabla, no podría usar los tokens.
- **Recuperación de contraseña:** el token sirve una sola vez y vence. Al cambiar la contraseña se cierran las sesiones abiertas.

Principios aplicados

Principio	Qué significa	Ejemplo en RutaAlerta
Mínimo privilegio	Cada actor ve sólo lo necesario	El afectado ve un sector aproximado, no la coordenada precisa
Defensa en profundidad	Varios controles independientes	La API filtra por organización y la base lo impide con claves foráneas compuestas
Separación de ámbitos	Los datos de una organización no se mezclan con otra	Un operador ajeno recibe error 403
Trazabilidad	Toda decisión deja actor, hora y motivo	Triggers impiden editar revisiones y auditoría
Privacidad por defecto	Lo más protegido es lo que viene activado	El trayecto se calcula en el teléfono; no se guarda el recorrido personal

Errores que no filtran información

Si un correo ya existe, la API responde «No fue posible registrar la cuenta con ese correo» sin revelar datos de la otra cuenta. Si un operador pide un caso que no existe o que es de otra organización, recibe el mismo

error 403. Así nadie puede adivinar qué casos existen.

Evidencia y archivos

Los archivos se guardan como objetos privados con una huella SHA-256. La huella detecta si el archivo cambió, pero no prueba que la escena sea verdadera. El original nunca se reemplaza por una versión mejorada o recortada.

Uso responsable

- Los aportes rechazados no se borran: se conservan con su estado y procedencia.
- Un rechazo no implica mala fe; una sanción exige motivo, revisión y derecho a reclamar.
- La trazabilidad técnica ayuda a revisar lo ocurrido, pero no es una cadena de custodia judicial.
- Antes de operar con datos reales habría que revisar la normativa de protección de datos y firmar acuerdos con las instituciones.

11. Base de datos I: conceptos desde cero

Este capítulo explica cada palabra que aparece en el modelo de datos. Los ejemplos salen del SQL real de RutaAlerta.

Qué es una base de datos

- **Base de datos:** conjunto ordenado de datos que se guardan de forma permanente y se consultan con reglas.
- **SGBD (sistema gestor de bases de datos):** el programa que guarda, protege y consulta los datos. RutaAlerta usa **PostgreSQL**.
- **Base relacional:** organiza los datos en tablas relacionadas entre sí por claves.
- **SQL:** el lenguaje para crear tablas (`CREATE TABLE`), guardar (`INSERT`), cambiar (`UPDATE`), borrar (`DELETE`) y consultar (`SELECT`).
- **Esquema:** carpeta lógica dentro de la base. Todas las tablas del proyecto viven en el esquema `rutaalerta`.

Tabla, fila y columna

Concepto	Significado	Ejemplo en RutaAlerta
Tabla (entidad)	Conjunto de cosas del mismo tipo	`caso`, `vehiculo`, `usuario`
Fila (registro, tupla)	Una cosa concreta	El caso de demostración con folio `DEMO-001`
Columna (atributo)	Una característica de esa cosa	`estado`, `fecha_hecho`, `zona`
Dominio	Valores permitidos de una columna	`estado` sólo admite reportado, en_revision, en_gestion o cerrado

Tipos de datos usados

Tipo	Qué guarda	Dónde aparece
`bigint`	Número entero grande	Todos los identificadores, como `id_caso`
`varchar(n)`	Texto con largo máximo n	`folio VARCHAR(30)`
`text`	Texto sin largo fijo	`descripcion_privada`
`timestamptz`	Fecha y hora con zona horaria	`fecha_hecho`, `creado_en`
`numeric` / `decimal`	Número decimal exacto	`precision_m`
`boolean`	Verdadero o falso	Preferencias activadas o no

Tipo	Qué guarda	Dónde aparece
`uuid`	Identificador único universal	`clave_envio` del caso
`bytea`	Bytes en bruto	`token_hash`, la huella de 32 bytes
`jsonb`	Documento JSON consultable	`payload` de `evento_salida`
`geography(Point,4326)`	Punto sobre la Tierra (PostGIS)	`ubicacion` del caso

Claves: cómo se identifica y se relaciona todo

- **Clave primaria (PK):** identifica una fila sin repetirse nunca. En RutaAlerta cada tabla tiene `id_<tabla>` como PK, por ejemplo `id_caso`.
- **Identidad (`GENERATED BY DEFAULT AS IDENTITY`):** la base asigna el número automáticamente: 1, 2, 3…
- **Clave foránea (FK):** columna que apunta a la PK de otra tabla. `caso.id_vehiculo` apunta a `vehiculo.id_vehiculo`. La base rechaza un caso con un vehículo que no existe. El modelo tiene **110** claves foráneas.
- **`ON DELETE RESTRICT`:** impide borrar una fila si otra la usa. No se puede borrar un vehículo que tiene casos.
- **Clave compuesta:** clave formada por dos o más columnas. `asignacion_caso` usa (`id_caso`, `id_organizacion`) para que sólo se asigne un operador de la misma organización del caso.
- **Clave natural frente a clave sustituta:** el folio o la patente son naturales (tienen sentido fuera del sistema); `id_caso` es sustituta (sólo sirve dentro de la base).

Restricciones: reglas que la base hace cumplir

Restricción	Qué hace	Ejemplo real
`NOT NULL`	La columna no puede quedar vacía	Todo caso tiene `fecha_hecho`
`NULL`	La columna puede no tener valor («se desconoce»)	`id_organizacion` del caso, si aún no hay receptor
`DEFAULT`	Valor que se pone si no se entrega otro	`estado` parte en `'reportado'`
`UNIQUE`	No se repite el valor	`folio` del caso; el correo normalizado del usuario
`CHECK`	Condición lógica obligatoria	Si el caso está cerrado, debe tener `motivo_cierre` y `cerrado_en`
Columna generada	Se calcula sola desde otra	`correo_normalizado = LOWER(TRIM(correo))`

Ejemplo de regla de negocio escrita como `CHECK`: si el motivo de cierre es «recuperado», el caso debe tener la fecha de recuperación y una evidencia que lo respalde (`ck_caso_24`).

Cardinalidad: cuántos con cuántos

Tipo	Lectura	Ejemplo
1 a 1	Uno se relaciona con uno	Un usuario tiene una sola fila de `preferencia`
1 a N	Uno se relaciona con muchos	Un caso tiene muchos avistamientos
N a M	Muchos con muchos	Usuarios y vehículos: se resuelve con la tabla intermedia `vinculo_vehiculo`

Una **tabla intermedia** (o asociativa) convierte una relación N a M en dos relaciones 1 a N y además guarda datos propios, como el tipo de vínculo o su vigencia.

Normalización

Normalizar es repartir los datos para que cada dato viva en un solo lugar y no se contradiga.

- **Primera forma normal (1FN):** cada celda guarda un solo valor; nada de listas dentro de una columna.
- **Segunda forma normal (2FN):** cada columna depende de la clave completa, no sólo de una parte.
- **Tercera forma normal (3FN):** ninguna columna depende de otra columna que no sea la clave.

Ejemplo: la marca no se escribe dentro de `vehiculo`. Existe `marca`, luego `modelo` (que apunta a la marca) y `vehiculo` apunta al modelo. Así «Toyota» se escribe una sola vez.

Otros objetos de la base

- **Índice:** estructura que acelera las búsquedas, como el índice de un libro. El modelo tiene 143 índices; 11 son espaciales (GiST).
- **Índice único parcial:** unicidad sólo para algunas filas. `uq_asignacion_caso_vigente` permite una sola asignación abierta (`fin IS NULL`) por caso, pero muchas cerradas en el historial.
- **Vista:** consulta guardada con nombre que se usa como si fuera una tabla. Ejemplo: `v_bandeja_institucional`. El modelo tiene 5.
- **Función:** código guardado en la base. Ejemplo: `revisar_avistamiento`, que guarda una decisión de forma completa. El modelo tiene 3.
- **Trigger (disparador):** función que se ejecuta sola ante un evento. `tr_auditoria_append` lanza un error si alguien intenta editar o borrar la auditoría. El modelo tiene 7.
- **Comentario (`COMMENT ON`):** descripción guardada dentro de la base para cada tabla.

Transacciones y ACID

Una **transacción** agrupa varios cambios que deben ocurrir todos o ninguno. Empieza con `BEGIN` y termina con `COMMIT` (confirmar) o `ROLLBACK` (deshacer).

Letra	Propiedad	Significado simple
A	Atomicidad	Todo o nada

Letra	Propiedad	Significado simple
C	Consistencia	Después del cambio se siguen cumpliendo todas las reglas
I	Aislamiento	Dos usuarios a la vez no se pisan
D	Durabilidad	Lo confirmado sobrevive a un corte de luz

Ejemplo: al revisar un avistamiento se guardan la decisión, el nuevo estado, el evento de salida y la auditoría. Si falla uno, no queda guardado ninguno.

Privilegios

`REVOKE ALL ... FROM PUBLIC` quita todos los permisos por defecto. Los usuarios finales nunca se conectan a la base: sólo la API lo hace.

12. Base de datos II: datos geográficos con PostGIS

RutaAlerta trabaja con lugares: dónde ocurrió un hecho, dónde se vio un vehículo y qué cubre una cámara. PostgreSQL por sí solo no entiende de mapas; para eso se usa **PostGIS**.

Conceptos básicos

- **SIG (sistema de información geográfica):** sistema que guarda y analiza datos con ubicación.
- **PostGIS:** extensión de PostgreSQL que agrega tipos y funciones geográficas. Se activa con `CREATE EXTENSION postgis`.
- **Latitud:** distancia al norte o sur del ecuador. Concepción está cerca de −36,82.
- **Longitud:** distancia al este u oeste de Greenwich. Concepción está cerca de −73,05.
- **WGS 84:** el sistema de coordenadas que usa el GPS.
- **SRID 4326:** el número que identifica a WGS 84. Toda geometría del modelo lo declara.

> Ojo: en PostGIS se escribe primero la longitud y después la latitud: `ST_Point(-73.05, -36.82)`. Es el error más común.

Geometry frente a geography

Tipo	Cómo calcula	Se usa en RutaAlerta para
`geography`	Sobre la Tierra curva; las distancias salen en metros	Puntos: ubicación del caso, del avistamiento y de la cámara
`geometry`	Sobre un plano; más rápido para cruces de formas	Líneas y polígonos: rutas, corredores y cobertura de cámaras

Formas que se guardan

Forma	Qué representa	Columna real
`Point`	Un lugar	`caso.ubicacion`, `avistamiento.ubicacion`, `camara.ubicacion`
`LineString`	Una línea, como un recorrido	`alternativa_ruta.geometria`
`MultiLineString`	Varias líneas	`hipotesis_ruta.geometria` (corredores posibles)
`MultiPolygon`	Una o varias áreas	`camara.cobertura` (lo que ve la cámara)

Funciones que hay que conocer

Función	Qué hace	Pregunta que responde
`ST_Point(lon, lat)`	Crea un punto	¿Dónde está?
`ST_SetSRID(g, 4326)`	Declara el sistema de coordenadas	¿En qué sistema está medido?
`ST_Distance(a, b)`	Distancia entre dos formas	¿A cuántos metros está?
`ST_DWithin(a, b, d)`	Verdadero si están a menos de d	¿Está dentro de 1 km?
`ST_Intersects(a, b)`	Verdadero si dos formas se tocan	¿El corredor pasa por lo que ve la cámara?
`ST_IsEmpty(g)`	Verdadero si la forma está vacía	Se usa en un `CHECK` para impedir ubicaciones vacías

Ejemplos reales (archivo 05_Consultas_espaciales.sql)

Cámaras a menos de 1 km de un punto, de la más cercana a la más lejana:

```
SELECT id_camara, nombre,
       ST_Distance(ubicacion, ST_SetSRID(ST_Point(-73.05,-36.82),4326)::geography) AS distancia_m
FROM camara
WHERE id_organizacion = 1 AND entorno = 'prueba' AND estado = 'disponible'
  AND ST_DWithin(ubicacion, ST_SetSRID(ST_Point(-73.05,-36.82),4326)::geography, 1000)
ORDER BY distancia_m;
```

Se lee así: crea el punto, mide la distancia a cada cámara, deja sólo las que están a 1000 metros o menos y ordénalas. Siempre se filtra por organización.

Corredores que cruzan la cobertura de una cámara: se une `hipotesis_ruta` con `camara` usando `ST_Intersects(h.geometria, c.cobertura)`.

Hechos validados por zona en el último año: un `GROUP BY zona` sobre `hecho_territorial`. Es para un mapa de calor interno; al ciudadano sólo se le muestra por sectores y con un mínimo de registros.

Índices espaciales

Un índice **GiST** acelera las preguntas espaciales. En vez de medir contra todas las cámaras, primero descarta las que están claramente lejos. El modelo tiene 11 índices GiST, por ejemplo `gis_caso_ubicacion` y `gis_camara_cobertura`.

Pruebas espaciales del modelo

- **T25:** una geometría sin SRID declarado es rechazada.
- **T29:** la distancia PostGIS se calcula en metros.
- **T30:** el cruce de geometrías funciona.

Límites que hay que decir en voz alta

- Un punto en el mapa es una observación en un momento; no es seguimiento continuo.
- Un corredor es una hipótesis, no una posición confirmada.
- La distancia o el cruce no son la probabilidad de encontrar el vehículo.

13. Base de datos III: el modelo de 56 tablas

Cifras para memorizar: 56 tablas, 549 columnas, 110 claves foráneas, 143 índices, 5 vistas, 3 funciones y 7 triggers, repartidos en 15 módulos dentro del esquema `rutaalerta`. Los diagramas del paquete agrupan esos módulos en 8 láminas (ER_01 a ER_08).

Los 15 módulos

N.º	Módulo	Tablas	Para qué sirve
01	Identidad (7)	`rol`, `usuario`, `usuario_rol`, `preferencia`, `consentimiento`, `dispositivo`, `token_acceso`	Cuentas, roles, permisos aceptados, teléfonos y sesiones
02	Organizaciones (2)	`organizacion`, `membresia`	Entidades receptoras y quién trabaja en cada una (operador o supervisor)
03	Vehículos (5)	`marca`, `modelo`, `vehiculo`, `foto_vehiculo`, `vinculo_vehiculo`	Ficha del vehículo y quién es propietario o conductor autorizado
04	Territorio consolidado (1)	`hecho_territorial`	Un hecho del territorio contado una sola vez, aunque lo informen varias fuentes

N.º	Módulo	Tablas	Para qué sirve
05	Casos (3)	`caso`, `asignacion_caso`, `gestion_caso`	El reporte privado, su responsable y la bitácora de gestiones
06	Red ciudadana (4)	`solicitud_colaboracion`, `avistamiento`, `revision_avistamiento`, `evidencia`	Pedir ayuda, recibir aportes, revisarlos y guardar archivos
07	Acompañamiento (2)	`acceso_confianza`, `recurso_apoyo`	Compartir el seguimiento con alguien de confianza y canales de apoyo
08	Fuentes históricas (3)	`fuente_datos`, `lote_importacion`, `registro_fuente`	Importar datos públicos con procedencia documentada
09	Trayectos (4)	`consulta_ruta`, `alternativa_ruta`, `trayecto`, `aviso_territorial`	Comparar rutas y dar avisos por cercanía
10	Notificaciones (3)	`notificacion`, `entrega_notificacion`, `evento_salida`	Avisos al usuario y cola de eventos
11	Control y evaluación (4)	`evento_auditoria`, `escenario_prueba`, `ejecucion_prueba`, `medicion`	Auditoría y medición de los objetivos con escenarios sintéticos
12	Moderación y derivación (3)	`moderacion_usuario`, `canal_derivacion`, `derivacion_caso`	Sanciones motivadas y envío de casos a canales institucionales
13	Cámaras y búsqueda (7)	`camara`, `sesion_busqueda`, `observacion_busqueda`, `hipotesis_ruta`, `recomendacion_camara`, `revision_recomendacion`, `consulta_video`	Búsqueda institucional con cámaras autorizadas
14	Asistencia con IA (6)	`version_modelo`, `ejecucion_modelo`, `candidato_visual`, `consulta_copiloto`, `afirmacion_copiloto`, `soporte_afirmacion`	Resultados de IA siempre revisables por una persona
15	Analítica (2)	`version_indicador`, `consulta_indicador`	Indicadores versionados y consultas reproducibles

> Los módulos 13 y 14 responden a OE5 y OE6 (CU26 a CU31). Están modelados, pero son una propuesta: los docentes aún no los ratifican.

El recorrido de un caso por las tablas

1. Ana tiene cuenta en `usuario` y su auto está en `vehiculo`, unido a ella por `vinculo_vehiculo`.
2. Ana reporta: se crea una fila en `caso` con estado `reportado` y una `clave_envio` única.
3. Un supervisor asigna el caso: fila en `asignacion_caso` con la membresía del operador.
4. Cada acción del operador agrega una fila en `gestion_caso` (nunca se edita).

5. Si se aprueba pedir ayuda, se crea `solicitud_colaboracion` con zona publicable y fecha de vencimiento.
6. Un vecino informa que vio el auto: fila en `avistamiento` con estado `pendiente`; si adjunta foto, va a `evidencia`.
7. El operador decide: `revisar_avistamiento` guarda la decisión en `revision_avistamiento`, actualiza el avistamiento, deja un evento en `evento_salida` y una línea en `evento_auditoria`.
8. El trabajador lee `evento_salida` y crea `notificacion` y `entrega_notificacion` para Ana.
9. Al cerrar, `caso` exige `motivo_cierre` y `cerrado_en`; si fue recuperado, también fecha y evidencia de respaldo.

Las 5 vistas

Vista	Qué muestra
`v_bandeja_institucional`	Casos con su asignación vigente, aportes pendientes y hora de la primera revisión
`v_solicitudes_publicables`	Solicitudes vigentes con patente, color, marca y modelo; sin dirección ni relato
`v_seguimiento_limitado`	Lo mínimo que ve el afectado: folio, estado, zona y fechas
`v_revisiones_oportunas`	Segundos entre la llegada de un aporte y su primera decisión
`v_casos_por_marca`	Casos y recuperaciones por mes, marca y modelo, sin contar duplicados

Las 3 funciones y los 7 triggers

Objeto	Qué hace
`actualizar_fecha()`	Pone la hora actual en `actualizado_en` al editar
`tr_actualizado` (3 triggers)	Usa la función anterior en `usuario`, `preferencia` y `caso`
`impedir_edicion_historial()`	Lanza un error si alguien intenta editar o borrar historial
4 triggers de historial	La aplican a `evento_auditoria`, `revision_avistamiento`, `gestion_caso` y `revision_recomendacion`
`revisar_avistamiento(...)`	Guarda una decisión completa en una sola transacción

Patrones que se repiten en el modelo

- **Historial de solo anexado:** las decisiones se agregan, no se reescriben.
- **Vigencia con `inicio` y `fin`:** si `fin` está vacío, la fila está vigente. Lo usan membresías, vínculos y asignaciones.
- **`clave_envio` única por autor:** si el teléfono reenvía lo mismo, no se duplica.
- **`entorno` prueba u operación:** separa datos de ensayo de datos reales; las métricas filtran por entorno.
- **Claves foráneas compuestas:** impiden mezclar datos de dos casos u organizaciones distintas.

- **Estado con reglas:** cada estado exige sus datos; por ejemplo, un avistamiento revisado exige revisor, hora y motivo.

14. Base de datos IV: el SQL comentado y cómo se verificó

Los archivos y su orden

Archivo	Qué hace
`01_Esquema_PostgreSQL_PostGIS.sql`	Crea la extensión, el esquema, las 56 tablas, claves, índices y triggers de fecha
`02_Vistas_y_operaciones.sql`	Crea las 5 vistas, los triggers de historial y la función `revisar_avistamiento`
`06_Catalogo_indicadores.sql`	Carga el catálogo de indicadores
`03_Datos_demo.sql`	Carga datos sintéticos (por ejemplo, folios `DEMO-001` y `DEMO-002`)
`04_Pruebas_integridad.sql`	Ejecuta las 36 pruebas de integridad
`05_Consultas_espaciales.sql`	Ejemplos de consultas PostGIS de solo lectura
`RutaAlerta_PostgreSQL_Completo.sql`	Une 01 y 02 en un solo archivo; no se ejecutan ambos en la misma base

Orden modular: 01 → 02 → 06 → 03 → 04 → 05. Orden completo: Completo → 06 → 03 → 04 → 05.

Leer un CREATE TABLE línea por línea

```
CREATE TABLE usuario (
    id_usuario bigint GENERATED BY DEFAULT AS IDENTITY NOT NULL, -- PK autonumérica
    nombre_visible VARCHAR(100) NOT NULL,                        -- obligatorio, máx. 100
    correo VARCHAR(254) NOT NULL,
    correo_normalizado VARCHAR(254)
        GENERATED ALWAYS AS (LOWER(TRIM(correo))) STORED,       -- se calcula solo
    password_hash VARCHAR(255) NULL,                             -- hash scrypt, nunca la clave
    correo_verificado_en timestamptz NULL,                       -- vacío = sin verificar
    estado text NOT NULL DEFAULT 'pendiente',
    creado_en timestamptz NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_en timestamptz NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_usuario),
    CONSTRAINT ck_usuario_0 CHECK (estado IN
        ('pendiente','activo','suspendido','anonimizado')),     -- dominio
    CONSTRAINT ck_usuario_1 UNIQUE (correo_normalizado),         -- un correo, una cuenta
    CONSTRAINT ck_usuario_2 CHECK (CHAR_LENGTH(TRIM(correo)) > 3)
);
```

Gracias a `correo_normalizado`, « PERSONA-A@EXAMPLE.INVALID » y «persona-a@example.invalid» cuentan como el mismo correo. La prueba T01 lo comprueba.

Una regla de negocio en SQL

```
CONSTRAINT ck_caso_20 CHECK (
  (estado = 'cerrado'  AND motivo_cierre IS NOT NULL AND cerrado_en IS NOT NULL)
  OR
  (estado <> 'cerrado' AND motivo_cierre IS NULL     AND cerrado_en IS NULL)
)
```

Se lee así: un caso cerrado debe decir por qué y cuándo; un caso abierto no puede tener esos datos.

La función revisar_avistamiento, paso a paso

1. Valida que la decisión sea aceptado, rechazado o requiere_aclaracion, y que el motivo no esté vacío.
2. Busca el avistamiento y lo **bloquea** (`FOR UPDATE`) para que dos operadores no decidan a la vez.
3. Comprueba que el revisor tenga una membresía activa en la organización del caso.
4. Inserta la decisión en `revision_avistamiento`.
5. Actualiza el estado del avistamiento.
6. Inserta un evento en `evento_salida` para avisar después.
7. Inserta una línea en `evento_auditoria`.

Como todo ocurre en una transacción, o se guardan las cuatro escrituras (pasos 4 a 7) o ninguna. El identificador del revisor debe venir de la sesión de la API, nunca de un formulario.

Cómo funcionan las pruebas de integridad

Cada prueba negativa intenta guardar algo prohibido y espera que la base lo rechace con un código concreto. Si la base lo acepta, la prueba falla.

Código SQLSTATE	Significado	Pruebas que lo esperan
`23505`	Se repitió un valor único	3
`23503`	Clave foránea a algo que no existe o que es de otro ámbito	8
`23514`	Se violó un `CHECK`	16
`22023`	Valor de parámetro inválido	1
`P0001`	Error lanzado por una función o trigger	2

Las otras 6 son positivas: comprueban que algo sí funciona, como la distancia en metros (T29), el cruce de geometrías (T30), que existan 56 tablas (T31) y 110 claves foráneas (T32) y la revisión completa con historial y evento (T33).

Ejemplos de pruebas negativas: T04, asignar un caso a una membresía de otra organización; T11, sobrescribir el historial de revisión; T20, un avistamiento con fecha futura; T22, cerrar un caso sin motivo ni

fecha.

Cómo se ejecutó la verificación

- **Herramienta:** PGlite 0.5.8 con PostGIS (pglite-postgis 0.2.8). Es PostgreSQL compilado a WebAssembly que corre dentro de Node.js, sin instalar un servidor.
- **Comandos:** `npm install` y luego `node run.mjs ../sql ../resultados completo` (o `modular`).
- **Resultado registrado el 14/10/2026:** 36 de 36 pruebas correctas por ambas rutas, en Windows 11 con Node.js v24.16.0.
- **Archivos de salida:** `resultado_completo.json`, `resultado_modular.json` y `checks_estaticos.json` (revisión de archivos y trazabilidad de las 56 tablas).

> Límite honesto: PGlite no reemplaza la prueba en un servidor PostgreSQL real ni las pruebas de concurrencia, rendimiento o restauración. El equipo debe repetir la ejecución y guardar su propio resultado.

15. La API inicial: cómo funciona por dentro

La API es el primer código ejecutable del proyecto. Cubre una parte del Sprint 1: HT01, HU01, HU03 y HU13. Está en `Evidencias de sistema/Aplicación/backend`.

Conceptos de API

- **API:** puerta por la que una aplicación pide o envía datos a otra. La app y el panel nunca tocan la base: le hablan a la API.
- **HTTP:** el protocolo de la web. Cada pedido tiene un método, una ruta, encabezados y a veces un cuerpo.
- **REST:** estilo de diseño donde cada ruta representa un recurso (`/casos`) y el método dice la acción.
- **Endpoint:** combinación de método y ruta, por ejemplo `POST /casos`.
- **JSON:** formato de texto con pares nombre-valor, como `{"folio": "RA-..."}`.
- **Token Bearer:** se envía en el encabezado `Authorization: Bearer <token>` para demostrar la sesión.
- **OpenAPI:** descripción automática de la API. FastAPI la muestra en `/docs`.

Métodos HTTP

Método	Uso	Ejemplo
`GET`	Leer	`GET /casos` lista la bandeja
`POST`	Crear	`POST /casos` crea un reporte
`PUT` / `PATCH`	Reemplazar o cambiar	Aún no se usan
`DELETE`	Borrar	No se usa: el historial no se borra

Códigos de respuesta que devuelve la API

Código	Significado	Cuándo aparece
200	OK	Un reintento devuelve el caso ya creado
201	Creado	Cuenta, sesión o caso nuevo
202	Aceptado	Se pidió recuperar la contraseña
204	Sin contenido	Contraseña cambiada
400	Pedido inválido	Token de recuperación vencido o ya usado
401	No autenticado	Falta el token o la sesión venció
403	Prohibido	Vehículo no vinculado o caso de otra organización
409	Conflicto	El correo ya está registrado

Código	Significado	Cuándo aparece
422	Datos no válidos	Fecha futura, latitud fuera de rango o clave no UUID

Los endpoints

Historia	Endpoint	Qué hace
HT01	`GET /health`	Comprueba la conexión, la versión de PostGIS y que existan las tablas
HU01	`POST /cuentas`	Registra un ciudadano con estado pendiente y rol ciudadano
HU01	`POST /sesiones`	Inicia sesión y entrega un token que dura 12 horas por defecto
HU01	`POST /cuentas/recuperacion` y `/confirmar`	Recuperación con token de un solo uso que vence en 30 minutos
HU03	`POST /casos`	Crea el reporte con folio, ubicación PostGIS y evento en la cola
HU13	`GET /casos` y `GET /casos/{id}`	Bandeja y detalle, sólo de la organización del operador

Las piezas del código

Archivo	Responsabilidad
`app/main.py`	Crea la aplicación FastAPI, une los routers y define `/health`
`app/config.py`	Lee la configuración desde variables de entorno (`DB_HOST`, `DB_PORT`, etc.)
`app/db.py`	Abre una conexión por pedido; confirma si todo salió bien y revierte si falló
`app/seguridad.py`	scrypt, huella SHA-256, emisión de tokens, usuario actual y ámbito por organización
`app/esquemas.py`	Modelos Pydantic que validan lo que entra y lo que sale
`app/routers/cuentas.py`	Registro, sesión y recuperación
`app/routers/casos.py`	Reporte, bandeja y detalle

- **Pydantic:** valida automáticamente. Si llega una latitud de 120, responde 422 sin tocar la base.
- **Dependencia (`Depends`):** FastAPI entrega a cada función la conexión y el usuario actual, ya revisados.
- **pg8000:** la librería de Python que conversa con PostgreSQL.

Qué pasa cuando se envía POST /casos

1. Revisa el token: si falta o venció, responde 401.
2. Revisa que `Idempotency-Key` sea un UUID.

3. Si ya existe un caso de ese usuario con esa clave, devuelve el mismo caso con 200 y `reintento: true`.
4. Rechaza una fecha futura con 422.
5. Comprueba que el vehículo esté vinculado a la cuenta; si no, 403.
6. Si viene organización, comprueba que esté activa.
7. Genera un folio como `RA-AAAAMMDD-XXXXXX`.
8. Inserta el caso con `ST_SetSRID(ST_Point(longitud, latitud), 4326)`.
9. En la misma transacción inserta el evento `caso_reportado` en `evento_salida`.
10. Responde 201 con id, folio, estado y hora.

Si dos envíos iguales llegan a la vez, la restricción única `ck_caso_17` frena al segundo, y la API devuelve el caso del primero.

Cómo ejecutarla y probarla

1. Crear el entorno: `python -m venv .venv` y activarlo.
2. Instalar: `pip install -r requirements.txt` y `npm install` en `scripts`.
3. Levantar la base de prueba: `node scripts/servidor_prueba_pglite.mjs "../../Base de datos/sql" 54329`.
4. En otra terminal: `pytest -v`.
5. Para ver la API: `uvicorn app.main:app --reload` y abrir `http://127.0.0.1:8000/docs`.

Resultado registrado el 14/10/2026: **10 pruebas aprobadas**, que corresponden a CP40 a CP44.

Lo que todavía no tiene

No hay app Flutter, panel React, verificación de correo, carga de evidencia ni despliegue en Azure. El código inicial se preparó con asistencia de IA desde el diseño del equipo: quien lo presente debe revisarlo, ejecutarlo y poder explicarlo.

16. Cómo se organiza el trabajo: Scrum

Scrum es un marco ágil: el trabajo se hace en ciclos cortos llamados *sprints* y al final de cada uno se muestra algo que funciona. En RutaAlerta cada sprint dura una semana.

Roles

Rol	Quién	Qué hace
Product Owner	Jorge Ceballos	Ordena el backlog y decide qué aporta más valor
Scrum Master	Alan Bravo	Facilita los eventos y quita obstáculos
Equipo de desarrollo	Jorge, Juan y Alan	Construyen y prueban el incremento

Los roles se acordaron el 14/10/2026 para empezar Scrum formal en el Sprint 1.

Eventos

Evento	Cuándo	Para qué	Documento
Sprint Planning	Inicio del sprint	Elegir historias y dividirlas en tareas con horas	05 Sprint Backlog
Daily Standup	Cada día hábil	¿Qué hice? ¿Qué haré? ¿Qué me bloquea?	06 Registro Daily
Sprint Review	Fin del sprint	Mostrar lo construido y recibir comentarios	08 Validación del Sprint
Retrospectiva	Después de la review	Mejorar la forma de trabajar	09 Retrospectiva

Artefactos

- **Product Backlog:** lista ordenada de todo lo que podría hacerse. RutaAlerta tiene 33 elementos (HU01 a HU31, HT01 y HT02) y 226 puntos.
- **Sprint Backlog:** lo comprometido para un sprint, dividido en tareas con horas.
- **Incremento:** lo terminado al final del sprint que cumple la Definition of Done.

Historias de usuario

Formato: **Como** [rol], **quiero** [funcionalidad], **para** [beneficio].

Ejemplo HU03: Como ciudadano afectado por un robo o intento, quiero reportar el incidente indicando qué ocurrió, dónde y cuándo, para obtener un folio y un estado que confirmen que mi reporte fue recibido.

- **Criterios de aceptación:** escenarios con la forma «En caso que… cuando… el sistema…». Dicen cuándo la historia está lista.
- **Historia técnica (HT):** trabajo necesario que no pide un usuario, como preparar el repositorio (HT01) o desplegar (HT02).
- **Épica:** grupo grande de historias. Hay 9: EP01 a EP09.

- **INVEST:** una buena historia es Independiente, Negociable, Valiosa, Estimable, pequeña (Small) y Testeable.

Estimación

- **Punto de historia:** mide esfuerzo y complejidad relativos, no horas. Se usa la serie de Fibonacci: 1, 2, 3, 5, 8, 13.
- **Planning Poker:** cada integrante muestra su estimación a la vez; si difieren, se conversa y se vuelve a votar.
- **Velocidad:** puntos terminados por sprint. Se conocerá después de cerrar sprints reales.
- **MoSCoW:** prioridad Must (debe), Should (debería), Could (podría) y Won't (no en esta entrega).

> Los puntos y horas actuales son estimaciones preliminares. Se validan en Planning Poker y Sprint Planning.

El plan de sprints

Sprint	Semana	Fechas	Historias	Puntos	Objetivo
1	S10	15 al 21/10	HT01, HU01, HU03, HU13	26	Cuenta, reporte con folio y bandeja del operador
2	S11	22 al 28/10	HU02, HU04, HU14, HU17	26	Vehículo, evidencia y revisión institucional
3	S12	29/10 al 4/11	HT02, HU05, HU06, HU07, HU11	26	Colaboración y seguimiento desplegados
4	S13	5 al 11/11	HU15, HU16, HU22	10	Piloto y estabilización
5	S14	12 al 18/11	Sin historias nuevas	0	Evaluación final, cierre y defensa

MVP: 26 + 26 + 26 + 10 = **88 puntos**. Los 138 puntos restantes quedan fuera del plan actual; incluyen la épica EP08 de cámaras e IA, marcada Won't.

Sprint 1 en detalle

- **Objetivo:** el ciudadano crea su cuenta, reporta un incidente y obtiene folio; el operador lo ve en la bandeja de su organización.
- **Tareas:** 15 tareas que suman 46 horas.
- **Capacidad supuesta:** 15 horas por integrante por semana, es decir 45 horas. El equipo debe confirmarla.
- **Ya hecho antes del sprint:** la API inicial con 10 pruebas aprobadas cubre parte de HT01, HU01, HU03 y HU13.
- **Falta:** Flutter, React, Docker, integración continua y la prueba en un servidor PostgreSQL nativo.

Burndown y Definition of Done

- **Burndown chart:** gráfico de horas restantes por día. La línea ideal baja en forma recta; la real se anota cada día.
- **Definition of Done (DoD):** lista que todo elemento debe cumplir para contar como terminado: código revisado, pruebas aprobadas, permisos verificados, documentación actualizada y demostrable.

> Los documentos 06 (Daily), 07 (Burndown), 08 (Review) y 09 (Retrospectiva) sólo se llenan con datos reales del equipo. No se inventan reuniones ni horas.

17. Pruebas: cómo se demuestra que funciona

Una **prueba** compara lo que el sistema hace con lo que debería hacer. Si coinciden, aprueba; si no, falla y se corrige. Todo está en el documento **C2 Plan de Pruebas y Evidencias**.

Niveles de prueba

Nivel	Qué prueba	Ejemplo en RutaAlerta
Unitaria	Una pieza aislada	La base rechaza un correo repetido (T01)
Integración	Varias piezas juntas	La API guarda el caso en la base con su folio (CP42)
Sistema	Todo el producto	Planificada cuando existan app y panel
Aceptación	Que cumpla lo que pidió el usuario	Recorrido completo en la Sprint Review (CP45, planificada)

Tipos según el objetivo

- **Funcional:** ¿hace lo que debe?
- **Seguridad:** ¿impide lo que no debe permitir? Ejemplo: un operador ajeno recibe 403.
- **Rendimiento:** ¿responde a tiempo con varios usuarios?
- **Positiva:** se entrega algo válido y se espera éxito.
- **Negativa:** se entrega algo prohibido y se espera rechazo.
- **Regresión:** se repiten las pruebas después de un cambio para ver que nada se rompió.

Partes de un caso de prueba

Campo	Ejemplo con CP43
Identificador	CP43
Tipo	Integración
Requisito que cubre	RF03 / HU03 · RNF03
Descripción	Un reintento con la misma clave idempotente no crea un segundo caso

Campo	Ejemplo con CP43
Resultado esperado	Primero 201, luego 200 con el mismo folio
Resultado obtenido	Aprobado: un solo caso

Los 54 casos de prueba

Bloque	Qué son	Estado
CP01 a CP36	Las 36 pruebas de integridad de la base (T01 a T36)	Ejecutadas: 36/36
CP37	La ruta modular y el script completo dan el mismo resultado	Ejecutada
CP38	Revisión de los SQL: sin credenciales, sin permisos a PUBLIC, sin DROP	Ejecutada, sin hallazgos
CP39	Trazabilidad: 31 casos de uso con objetivo y 56 tablas asociadas	Ejecutada, sin huérfanos
CP40 a CP44	Pruebas de la API con pytest	Ejecutadas: 10/10
CP45 a CP54	Aceptación, rendimiento, seguridad, interrupciones y respaldo	Planificadas

Total: 44 ejecutadas y 10 planificadas. Todas las ejecutadas usan datos sintéticos.

Las pruebas de la API (CP40 a CP44)

Caso	Qué comprueba	Resultado registrado
CP40	Registro válido; correo repetido; registro sin ámbito institucional	201; 409 sin revelar la otra cuenta; 403 en la bandeja
CP41	Token de recuperación de un solo uso y con vencimiento	Primer uso 204; reutilizado 400; vencido 400
CP42	El reporte devuelve folio, estado y fecha	201, folio RA-…, ubicación PostGIS y evento registrados
CP43	Reintento sin duplicar	201 y luego 200 con el mismo folio
CP44	Aislamiento por organización	Bandeja filtrada; 403 al operador ajeno

Además: `/health` conecta con PostGIS, sin token no hay acceso y el reporte rechaza vehículos no vinculados y fechas futuras.

Herramientas

- **pytest:** ejecuta las pruebas de Python y muestra PASSED o FAILED.
- **Fixture:** preparación reutilizable, como crear un usuario de prueba antes de cada caso. Están en `tests/conftest.py`.
- **PGlite:** PostgreSQL en WebAssembly para probar sin instalar un servidor.
- **Planificadas:** k6 para rendimiento (CP46), OWASP ZAP y auditoría de dependencias para seguridad (CP47).

Pruebas pendientes importantes

- **CP46:** `GET /casos` con 10 usuarios a la vez, con el 95 % de las respuestas bajo 1 segundo.
- **CP49:** 20 interrupciones controladas sin perder ni duplicar operaciones.
- **CP50:** restaurar un respaldo en menos de dos horas.
- **CP51 a CP54:** ensayos de aceptación para medir OE1 a OE4.

> Regla: una prueba planificada no se presenta como ejecutada. Los resultados futuros se anotan con fecha, entorno y responsable reales.

18. Indicadores M01 a M18: cómo se mide

Un **indicador** es un número definido con reglas fijas para saber cómo va algo. RutaAlerta define 18 (M01 a M18). Su detalle está en `anexos/Indicadores_y_datos.md` y sus definiciones se cargan con `06_Catalogo_indicadores.sql`.

> Estado actual: las definiciones están cargadas en SQL. El cálculo automático y el panel están pendientes. Ningún indicador tiene todavía resultados reales.

Conceptos de medición

Concepto	Significado	Ejemplo
Numerador	Lo que se cuenta arriba de la fracción	Casos con recuperación respaldada
Denominador	El total contra el que se compara	Casos elegibles de esa cohorte
Tasa o porcentaje	Numerador dividido por denominador	3 de 12 = 25 %
Mediana	El valor del medio al ordenar los datos; no se deja arrastrar por extremos	Minutos hasta la primera revisión
Percentil 95 (p95)	El 95 % de los casos está por debajo de este valor	p95 de tiempo de revisión
Cohorte	Grupo que comparte un período de inicio	Casos reportados en noviembre
Fecha de corte	Momento en que se toma la foto de los datos	Corte al 18/11 a las 12:00
Sesgo de selección	Error por mirar sólo un subgrupo	Medir tiempo de recuperación sólo entre recuperados
Versión del indicador	La definición queda fechada; si cambia, se crea otra versión	Tabla `version_indicador`

Regla clave: «sin datos» no es cero. Si el denominador es cero, no se calcula el porcentaje; el estado queda `sin_datos`.

Los 18 indicadores

Id	Nombre	Qué mide, en simple
M01	Casos por sector	Cuántos casos únicos hay por zona, separando pendientes y duplicados
M02	Evolución mensual y anual	Cómo cambian los casos entre períodos comparables
M03	Horarios y días	A qué hora y día ocurren, en hora de Chile
M04	Marca, modelo y tipo	Casos según características del vehículo
M05	Tipo de incidente	Encerrona, portonazo, robo estacionado u otro
M06	Concentración territorial	Dónde se agrupan los hechos, sin publicar puntos privados
M07	Cobertura de datos	Qué parte de los registros tiene fecha, zona y precisión suficientes
M08	Aportes por caso	Avistamientos recibidos, revisados, aceptados y pendientes
M09	Primera revisión	Mediana y p95 de minutos hasta la primera revisión humana
M10	Carga y antigüedad	Casos abiertos por responsable y por antigüedad; no es un ranking laboral
M11	Recuperaciones documentadas	Casos con recuperación respaldada sobre casos elegibles de la cohorte
M12	Tiempo hasta recuperación	Tiempo entre el hecho y la recuperación, sólo entre recuperados
M13	Calidad y moderación	Duplicados, rechazados y abusos confirmados, por separado
M14	Evidencia y notificaciones	Cargas exitosas y entregas confirmadas por canal
M15	Cobertura de cámaras	Incidentes de prueba con al menos una cámara autorizada y utilizable
M16	Búsqueda asistida	Tiempo hasta una observación correcta, comparado con búsqueda manual
M17	Calidad de candidatos	Consultas con el candidato correcto entre los tres primeros
M18	Apoyo y autonomía	Recorridos de seguimiento completados y derivaciones

M15 a M17 dependen de cámaras e IA (OE5, propuesto). M16 y M17 usan una verdad de referencia fijada antes del ensayo.

Advertencias que vienen con los indicadores

- **M01 a M06** describen reportes, no la criminalidad real: sólo cuentan lo que se informó.
- **M10** no sirve para comparar el desempeño de personas sin considerar complejidad y turnos.
- **M12** sólo mira a los recuperados; por eso se aclara el sesgo.
- **M13:** un aporte rechazado no significa que la persona mintiera.
- **M14:** que el proveedor acepte un push no prueba que el usuario lo leyó.
- **M18** mide tareas y derivaciones, no la recuperación emocional.

Cómo se guarda una consulta de indicador

Cada consulta queda en `consulta_indicador` con la versión usada, el período, los filtros, el numerador, el denominador, el resultado y el estado (`calculado`, `sin_datos` o `error`). Así cualquier persona puede repetir el cálculo y obtener el mismo número.

19. Los documentos de la Fase 2 y el repositorio

Qué pidió el docente (ruta ágil)

- Plantillas 01 a 04 de la metodología ágil.
- Del DAS (C1): el modelo de datos (4.2.4), los diagramas UML de 4.1, 4.2, 4.3 y 4.4.4, y los requisitos no funcionales (2.4 y 5).
- Evidencias individuales (2.1, un diario por integrante) y grupales (2.4).
- Todo se sube a `fase 2/EVIDENCIAS PROYECTO/` y sus carpetas hermanas. Todas las plantillas son primeras versiones hasta el final.

Qué contiene cada documento

Documento	Qué tiene	Para qué sirve
01 Visión y Roles Scrum	Problema, visión, alcance, roles PO y SM	Explicar qué se construye y quién decide
02 Épicas e Historias	9 épicas, 31 HU y 2 HT con criterios de aceptación	Describir el producto desde el usuario
03 Plantilla de Historias (xlsx)	Las historias en formato de planilla	Registro editable de historias
04 Product Backlog (xlsx)	Prioridad, puntos (226) y sprint propuesto	Ordenar el trabajo
05 Sprint Backlog (xlsx)	15 tareas del Sprint 1 con horas (46) y responsables	Plan del sprint
06 Registro Daily (xlsx)	Plantilla para anotar cada daily	Se llena sólo con reuniones reales
07 Burndown (xlsx)	Horas restantes por día	Se llena con horas reales
08 Sprint Review	Datos del sprint y lo comprometido	Se completa al cerrar el sprint
09 Retrospectiva	Formato de mejora	Se completa al cerrar el sprint
10 Definition of Done	Criterios para dar algo por terminado	Calidad mínima común
C1 DAS	Arquitectura, modelo 4+1, requisitos, diagramas UML	Documento técnico principal
C2 Plan de Pruebas	CP01 a CP54, ejecutadas y planificadas	Evidencia de calidad

Documento	Qué tiene	Para qué sirve
C3 Manual Técnico	Cómo instalar, ejecutar y desplegar	Que otra persona pueda levantar el sistema
Libro v4.1 (pdf)	Documento base del proyecto	Antecedente del diseño
2.1 Diarios	Reflexión individual de la fase	Cada integrante debe revisar y firmar el suyo
2.4 Guía del estudiante	Avance grupal de la fase	Evidencia grupal
Presentación	Diapositivas para 10 minutos	Defensa oral

Las secciones del DAS que pidió el docente

Sección	Contenido
2.4	Requerimientos no funcionales RNF01 a RNF12
4.1	Vista de escenario: actores y diagramas de casos de uso
4.2	Vista lógica: diagrama de clases y modelo de datos (4.2.4)
4.3	Vista de implementación: diagramas de componentes y de paquetes
4.4.4	Diagrama de secuencia de la funcionalidad principal
5	Requisitos de calidad: atributos y criterios de evaluación

La estructura en GitHub

Repositorio: `github.com/AlanBravo19/Capstone_2026_G7`.

```
fase 2/
├── Evidencias Individuales/   (3 diarios 2.1)
├── Evidencias Grupales/       (2.4 y planilla de avance)
└── EVIDENCIAS PROYECTO/
    ├── Presentación Proyecto.pptx y .pdf
    ├── Evidencias de documentación/   (01–10, C1–C3, libro, diagramas)
    └── Evidencias de sistema/
        ├── Aplicación/        (backend y resultados de pytest)
        └── Base de datos/     (sql, diagramas, anexos, verificación)
```

Quién sube qué

Integrante	Responsabilidad
Jorge Ceballos	01 a 04, libro, diagramas, presentación, 2.4, planilla y su diario
Alan Bravo	Evidencias de sistema (Aplicación y Base de datos), C1 a C3 y su diario
Juan Marinan	05 a 10 y su diario

Cada uno sube desde su propia cuenta lo que le corresponde. Para ver el estado actual, revisar el repositorio en GitHub.

Conceptos de Git y GitHub

Concepto	Significado
Repositorio	Carpeta con todo el historial de cambios del proyecto
Commit	Una foto de los cambios con autor, fecha y mensaje
Push	Enviar los commits locales a GitHub
Pull	Traer a tu computador los cambios de GitHub
Rama (branch)	Línea de trabajo paralela; `main` es la principal
Pull request	Pedir que se revisen y unan cambios de una rama a otra
`.gitignore`	Lista de archivos que no se suben, como `node_modules` o `.env`
README	Archivo que explica la carpeta al abrirla en GitHub

Mensajes de commit con prefijo: `feat:` (función nueva), `docs:` (documentación), `fix:` (corrección) y `chore:` (orden o mantenimiento).

Qué no se sube

- Las carpetas `NO_SUBIR_uso_interno` y `NO_SUBIR_aun_entrega_final` (2.6 y C4 a C6 en borrador).
- `node_modules`, `.env` o cualquier contraseña.

20. Preguntas probables en la defensa

Cada respuesta está pensada para decirse en menos de 30 segundos. Léela, ciérrala y respóndela con tus palabras.

Sobre el proyecto

1. ¿Qué es RutaAlerta en una frase? Una plataforma colaborativa que ayuda a ciudadanos e instituciones a reportar, revisar y dar seguimiento a robos de vehículos con información ordenada, oportuna y trazable.

2. ¿Qué problema resuelve? Hoy la información del robo queda dispersa en mensajes, redes y archivos. Nadie la ordena, no se sabe quién la revisó y el afectado no sabe en qué va su caso.

3. ¿Cuál es el objetivo general? Contribuir a una respuesta ciudadana e institucional más informada, oportuna y trazable frente al robo vehicular, evaluada en un piloto al término de las 14 semanas.

4. ¿Cómo saben si cumplieron? Cada objetivo específico tiene una meta medible. Ejemplo: OE2 busca reducir al menos 20 % la mediana del tiempo del operador para reunir antecedentes, frente a un procedimiento con archivos separados, con al menos 12 pares de ensayos.

5. ¿Qué pasa con OE5 y OE6? Son una ampliación propuesta en la versión 4.1. Están modeladas, pero no las comprometemos hasta que los docentes las ratifiquen.

Sobre los límites

6. ¿Predicen dónde ocurrirá un robo? No. Mostramos antecedentes de reportes pasados con su fecha y procedencia. No es una predicción.

7. ¿Garantizan una ruta segura? No. Comparamos alternativas y explicamos por qué; el puntaje es relativo y no es una probabilidad de delito.

8. ¿Pueden seguir el auto robado con el GPS del celular? No. El GPS del teléfono de la víctima marca dónde está el teléfono, no el vehículo. Después del robo sólo hay observaciones fechadas de personas o cámaras autorizadas.

9. ¿Tienen convenio con la municipalidad o Carabineros? No. El sistema está preparado para organizaciones receptoras, pero el piloto usa organizaciones de demostración. Registrar un canal no crea un convenio.

10. ¿Reemplaza la denuncia? No. El caso es un reporte privado. El sistema invita a denunciar y guarda si se declaró una denuncia, pero no la reemplaza.

Sobre la técnica

11. ¿Qué tecnologías usan y por qué? Flutter para la app (un código para Android e iOS), React para el panel, FastAPI en Python para la API y PostgreSQL con PostGIS para datos geográficos. Azure para el despliegue del piloto.

12. ¿Por qué la app no se conecta directo a la base? Porque la API es la única puerta: ahí se valida, se autoriza por organización y se audita.

13. ¿Cómo evitan casos duplicados si se corta la señal? Con idempotencia: cada envío lleva una `Idempotency-Key`. Si llega dos veces, la API devuelve el mismo caso. Lo prueba CP43.

14. ¿Cómo guardan las contraseñas? Con scrypt y sal aleatoria. Los tokens de sesión se guardan sólo como huella SHA-256.

15. ¿Qué es la cola transaccional? El aviso se guarda en `evento_salida` en la misma transacción que el cambio. Si el cambio no se confirma, el aviso tampoco existe. Un trabajador lo envía después.

Sobre la base de datos

16. ¿Cuántas tablas tiene y cómo lo verificaron? 56 tablas, 549 columnas y 110 claves foráneas. Lo verificamos con 36 pruebas de integridad en PGlite con PostGIS: 36 de 36 correctas el 14/10/2026.

17. ¿Cómo impiden que una organización vea casos de otra? En dos capas: la API filtra por membresía activa y la base usa claves foráneas compuestas. Un operador ajeno recibe 403 (CP44).

18. ¿Por qué no se puede editar el historial? Porque las decisiones deben poder revisarse. Un trigger impide editar o borrar la auditoría, las gestiones y las revisiones.

Sobre el trabajo en equipo

19. ¿Qué tienen funcionando hoy? El modelo de datos verificado y una API inicial con registro, sesión, recuperación, reporte con folio y bandeja, con 10 pruebas aprobadas. Aún no hay app, panel ni despliegue.

20. ¿Cómo se organizan? Scrum semanal: Jorge es Product Owner y Alan Scrum Master. El MVP suma 88 puntos en 4 sprints, más un sprint de cierre.

21. ¿Usaron inteligencia artificial? Sí, como asistencia para preparar documentos y código inicial desde nuestro diseño. Lo revisamos, lo ejecutamos y lo declaramos en cada README.

Consejo final

Si no sabes algo, dilo y explica cómo lo averiguarías. Nunca inventes un resultado, una reunión o un convenio.

21. Glosario de la A a la Z

Término	Significado
ACID	Atomicidad, consistencia, aislamiento y durabilidad: las garantías de una transacción
Afectado	Persona que sufrió el robo o intento y reporta el caso
API	Puerta por la que la app y el panel piden o envían datos; la única que toca la base
Auditoría	Registro de quién hizo qué y cuándo; en RutaAlerta no se puede editar
Autenticación	Comprobar quién eres
Autorización	Decidir qué puedes hacer

Término	Significado
Avistamiento	Aporte de un ciudadano que dice haber visto el vehículo en un lugar y hora
Azure	Nube de Microsoft elegida para el piloto (Blob, Container Apps, Key Vault)
Backlog	Lista ordenada de trabajo pendiente
Bandeja institucional	Lista de casos que ve el operador de una organización
Burndown	Gráfico de horas restantes del sprint
Caso	Reporte privado de un robo o intento; no es una denuncia
Caso de uso (CU)	Descripción de cómo un actor usa el sistema para lograr algo
CHECK	Regla lógica que la base obliga a cumplir
Clave foránea (FK)	Columna que apunta a la clave primaria de otra tabla
Clave primaria (PK)	Columna que identifica cada fila sin repetirse
Cola transaccional (outbox)	Guardar el evento en la misma transacción del cambio para enviarlo después
Commit	Foto de cambios en Git; también, confirmar una transacción en SQL
Contacto de confianza	Persona a quien el afectado da acceso limitado y revocable a su caso
Criterio de aceptación	Condición que dice cuándo una historia está lista
Daily Standup	Reunión diaria breve del equipo
DAS	Documento de Arquitectura de Software (C1)
Definition of Done (DoD)	Criterios para considerar algo terminado
Endpoint	Método más ruta de la API, como `POST /casos`
Épica	Grupo grande de historias de usuario
Esquema	Carpeta lógica de la base; aquí, `rutaalerta`
Evidencia	Foto, video o documento asociado a un caso, guardado en privado con su huella
FastAPI	Framework de Python para construir la API
Flutter	Framework de Google para la app móvil Android e iOS
Folio	Código visible que identifica un caso, como `RA-20261015-A1B2C3`
Geography / Geometry	Tipos PostGIS: sobre la Tierra (metros) o sobre un plano
GiST	Tipo de índice para datos espaciales
Hash / huella	Resumen irreversible de un dato; SHA-256 produce 32 bytes
Hecho territorial	Un hecho del territorio consolidado y contado una sola vez

Término	Significado
Historia de usuario (HU)	Necesidad escrita como «Como… quiero… para…»
Historia técnica (HT)	Trabajo necesario que no pide un usuario
HTTP	Protocolo de la web: método, ruta, encabezados y cuerpo
Idempotencia	Repetir una operación da el mismo resultado sin duplicar
Índice	Estructura que acelera búsquedas
Incremento	Lo terminado al final de un sprint
Indicador	Número con reglas fijas para medir algo (M01 a M18)
JSON / JSONB	Formato de datos nombre-valor; JSONB es su versión consultable en PostgreSQL
Key Vault	Servicio de Azure para guardar secretos fuera del código
Latitud / longitud	Posición norte-sur y este-oeste; en PostGIS va primero la longitud
Mínimo privilegio	Dar a cada actor sólo el acceso necesario
Membresía	Relación de un usuario con una organización y su rol (operador o supervisor)
MER	Modelo entidad-relación: el diseño conceptual de la base
MoSCoW	Prioridad Must, Should, Could y Won't
MVP	Producto mínimo viable: 88 puntos en RutaAlerta
Normalización	Repartir los datos para que cada uno viva en un solo lugar
NULL	Ausencia de valor («se desconoce»), distinto de cero o vacío
OE	Objetivo específico (OE1 a OE6)
OpenAPI	Descripción automática de la API, visible en `/docs`
Operador	Persona de una organización que revisa y gestiona casos
Organización receptora	Entidad que recibe y gestiona casos en su territorio
PGlite	PostgreSQL en WebAssembly, usado para probar sin servidor
Planning Poker	Técnica de estimación por votación simultánea
PostGIS	Extensión geográfica de PostgreSQL
PostgreSQL	Sistema gestor de base de datos relacional del proyecto
Product Owner	Rol que ordena el backlog; Jorge Ceballos
Punto de historia	Medida relativa de esfuerzo y complejidad
Pydantic	Librería que valida los datos que entran y salen de la API
React	Librería para construir el panel web institucional

Término	Significado
REST	Estilo de API basado en recursos y métodos HTTP
RF / RNF	Requisito funcional (qué hace) y no funcional (cómo de bien lo hace)
Rollback	Deshacer una transacción
scrypt	Algoritmo lento a propósito para guardar contraseñas
Scrum Master	Rol que facilita el proceso; Alan Bravo
Solicitud de colaboración	Publicación aprobada, con vencimiento, que pide ayuda a la red ciudadana
Sprint	Ciclo de trabajo de una semana
SQL	Lenguaje para crear y consultar bases relacionales
SQLSTATE	Código de error estándar de la base, como 23505 por valor repetido
SRID 4326	Identificador del sistema de coordenadas WGS 84 del GPS
Supervisor	Rol de organización que asigna casos y autoriza publicaciones
Token	Cadena secreta que demuestra una sesión; se guarda sólo su huella
Trabajador (worker)	Proceso aparte que lee la cola de eventos y envía avisos
Transacción	Grupo de cambios que ocurren todos o ninguno
Trazabilidad	Poder seguir el vínculo entre objetivo, requisito, historia, tabla y prueba
Trigger	Función que la base ejecuta sola ante un evento
UML	Lenguaje de diagramas: casos de uso, clases, componentes, secuencia y despliegue
UUID	Identificador único universal de 128 bits
Vínculo vehicular	Relación revisable entre un usuario y un vehículo (propietario o conductor)
Vista	Consulta guardada que se usa como una tabla
Vista 4+1	Modelo de arquitectura con vistas de escenario, lógica, implementación, procesos y física

22. Autoevaluación

Responde sin mirar y después revisa las soluciones. Si fallas una, vuelve al capítulo indicado.

Parte A: el proyecto

1. ¿Qué diferencia hay entre un caso de RutaAlerta y una denuncia? *(cap. 2)*
2. Nombra los cuatro estados de un caso en orden. *(cap. 6)*
3. ¿Qué objetivos están comprometidos y cuáles sólo propuestos? *(cap. 3)*

4. ¿Por qué el GPS del teléfono de la víctima no sirve para seguir el vehículo? *(cap. 2)*
5. ¿Qué significa que RF03 = CU03 = HU03? *(cap. 7)*

Parte B: base de datos

6. ¿Qué es una clave foránea? Da un ejemplo real. *(cap. 11)*
7. ¿Qué hace `ON DELETE RESTRICT`? *(cap. 11)*
8. ¿Qué permite el índice `uq_asignacion_caso_vigente`? *(cap. 11)*
9. ¿Por qué en PostGIS se escribe `ST_Point(-73.05, -36.82)` y no al revés? *(cap. 12)*
10. ¿Cuándo se usa `geography` y cuándo `geometry`? *(cap. 12)*
11. ¿Qué pasa si alguien intenta editar `evento_auditoria`? *(cap. 13)*
12. ¿Qué significa el código SQLSTATE 23514? *(cap. 14)*
13. ¿Cuántas escrituras hace `revisar_avistamiento` y por qué van en una transacción? *(cap. 14)*

Parte C: API, seguridad y pruebas

14. ¿Qué código devuelve `POST /casos` la primera vez y cuál en un reintento con la misma clave? *(cap. 15)*
15. ¿Por qué un operador ajeno recibe 403 tanto si el caso existe como si no? *(cap. 10)*
16. ¿Qué se guarda en `token_acceso`: el token o su huella? *(cap. 10)*
17. ¿Cuántos casos de prueba hay, cuántos están ejecutados y cuántos planificados? *(cap. 17)*
18. ¿Qué diferencia hay entre una prueba unitaria y una de integración? *(cap. 17)*

Parte D: Scrum e indicadores

19. ¿Qué historias tiene el Sprint 1 y cuántos puntos suman? *(cap. 16)*
20. ¿Cuántos puntos tiene el MVP y cuántos el backlog total? *(cap. 16)*
21. ¿Por qué «sin datos» no es lo mismo que cero en un indicador? *(cap. 18)*
22. ¿Qué sesgo tiene M12? *(cap. 18)*

Soluciones

1. El caso es un reporte privado dentro de la plataforma; la denuncia es un trámite oficial. El caso no la reemplaza.
2. Reportado, en revisión, en gestión y cerrado.
3. Comprometidos: OE1 a OE4. Propuestos y sin ratificar: OE5 y OE6.
4. Porque marca dónde está el teléfono. Si el teléfono no va en el auto, no dice nada del vehículo.
5. Que el requisito, el caso de uso y la historia tienen el mismo número y se pueden seguir entre sí.
6. Una columna que apunta a la clave primaria de otra tabla. Ejemplo: `caso.id_vehiculo` apunta a `vehiculo.id_vehiculo`.
7. Impide borrar una fila si otra fila la está usando.
8. Sólo una asignación abierta por caso, pero muchas cerradas en el historial.
9. Porque PostGIS recibe primero la longitud (x) y después la latitud (y).

10. `geography` para puntos y distancias en metros; `geometry` para líneas y polígonos que se cruzan.
11. El trigger `tr_auditoria_append` lanza un error y el cambio no se guarda.
12. Que se violó una regla `CHECK`.
13. Cuatro: revisión, estado del avistamiento, evento de salida y auditoría. Van juntas para que no quede una a medias.
14. 201 la primera vez; 200 con el mismo folio en el reintento.
15. Para no revelar qué casos existen.
16. Sólo la huella SHA-256 de 32 bytes.
17. 54 en total: 44 ejecutados y 10 planificados.
18. La unitaria prueba una pieza aislada; la de integración prueba varias piezas juntas, como la API con la base.
19. HT01, HU01, HU03 y HU13: 26 puntos.
20. MVP: 88 puntos. Backlog total: 226 puntos.
21. Porque cero es un resultado medido; «sin datos» significa que no hay base para calcular.
22. Sesgo de selección: sólo mide a los vehículos recuperados.

Cierre

Si respondiste bien 18 o más, estás listo para explicar el proyecto. Para la defensa, repasa el capítulo 20 en voz alta y ensaya con tus compañeros.

Anexo A. Mockups de todas las pantallas

> Estos mockups son una **propuesta de diseño para estudiar y discutir**. No son un prototipo implementado ni se probaron con usuarios. Los datos que aparecen (folios, patentes, fechas, nombres) son ficticios. El equipo debe validarlos antes de construir las pantallas.

Qué es un mockup

Término	Qué es	Para qué sirve
Wireframe	Esquema en blanco y negro con cajas y textos	Decidir qué va en cada pantalla
Mockup	Dibujo estático más detallado, con textos reales y estados	Explicar cómo se verá y qué reglas muestra
Prototipo	Versión navegable, con clics entre pantallas	Probar con usuarios antes de programar

Este anexo trae mockups. El siguiente paso sería un prototipo navegable, por ejemplo en Figma, para probarlo con usuarios.

Cómo leerlos

- Cada dibujo muestra una o dos pantallas.
- Los **círculos con número** señalan un elemento. La tabla de abajo explica qué hace y qué regla del proyecto cumple.
- Los colores tienen significado: verde es un estado correcto o confirmado, amarillo es pendiente o advertencia, rojo es error o quitar algo, y azul es la acción principal.

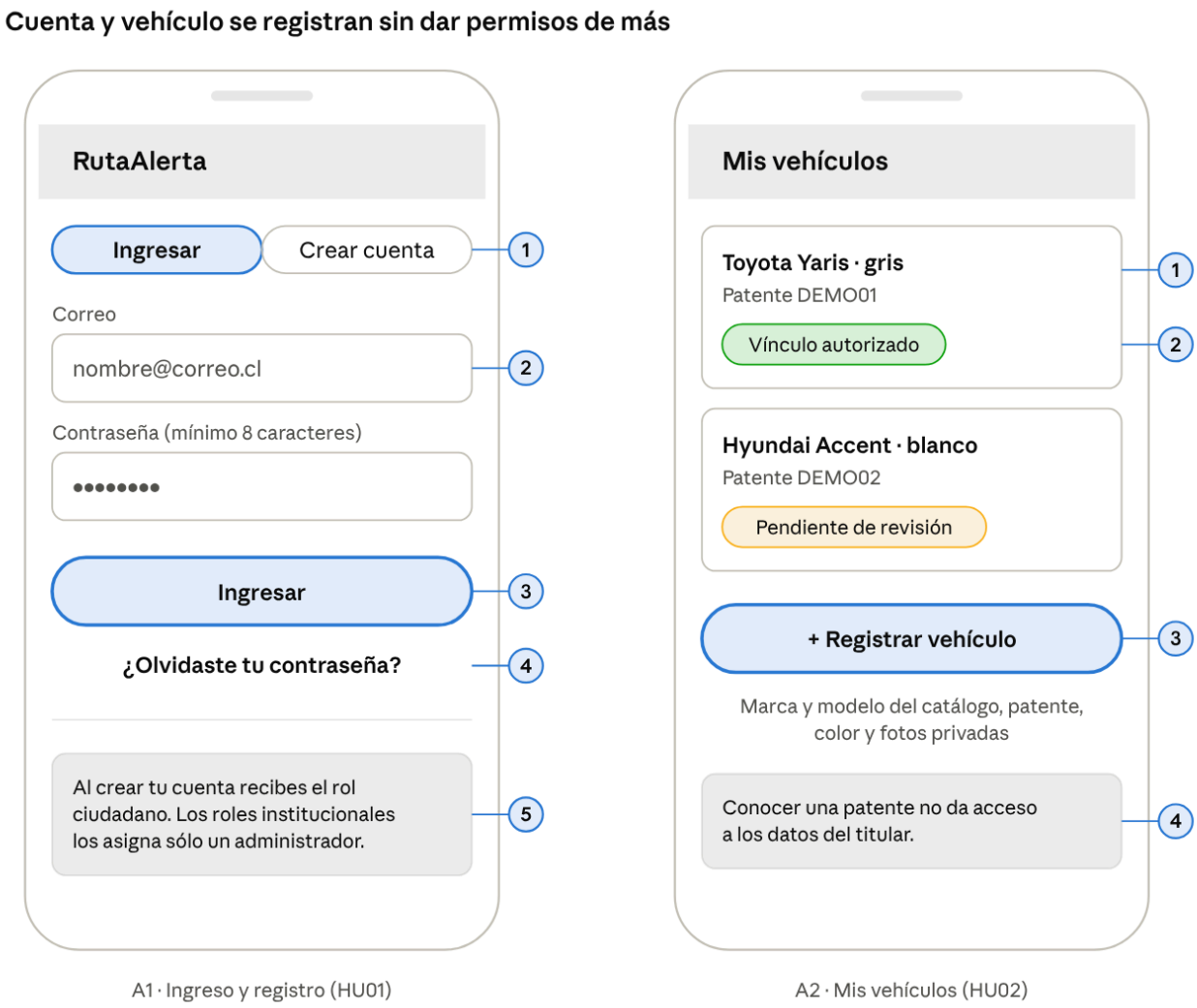
Mapa de pantallas

Código	Pantalla	Quién la usa	Historias	Prioridad
A1	Ingreso y registro	Ciudadano	HU01	Must, Sprint 1
A2	Mis vehículos	Ciudadano	HU02	Must, Sprint 2
A3	Reportar incidente	Afectado	HU03	Must, Sprint 1
A4	Reporte recibido y evidencia	Afectado	HU03, HU04	Must, Sprints 1 y 2
A5	Mi caso	Afectado	HU06	Must, Sprint 3
A6	Contacto de confianza	Afectado	HU07	Must, Sprint 3
A7	Solicitudes vigentes	Colaborador	HU11	Must, Sprint 3
A8	Informar avistamiento	Colaborador	HU05, HU04	Must, Sprint 3

Código	Pantalla	Quién la usa	Historias	Prioridad
A9	Antecedentes y rutas	Ciudadano	HU08, HU09	Should y Could, por acordar
A10	Avisos y privacidad	Ciudadano	HU10, HU12, HU25	Should y Could, por acordar
P1	Bandeja de casos	Operador	HU13	Must, Sprint 1
P2	Detalle del caso	Operador y supervisor	HU15, HU16	Must, Sprint 4
P3	Revisión de avistamientos	Operador	HU14	Must, Sprint 2
P4	Solicitud de colaboración	Operador	HU17	Must, Sprint 2
P5	Indicadores	Operador	HU18	Should, por acordar
P6	Administración	Administrador	HU19 a HU24	Should y Must (HU22)
P7	Búsqueda asistida	Operador	HU26 a HU31	Won't: propuesta OE5

Las pantallas A son de la **app móvil** (Flutter). Las pantallas P son del **panel web** (React).

A.1 App móvil: cuenta y vehículos

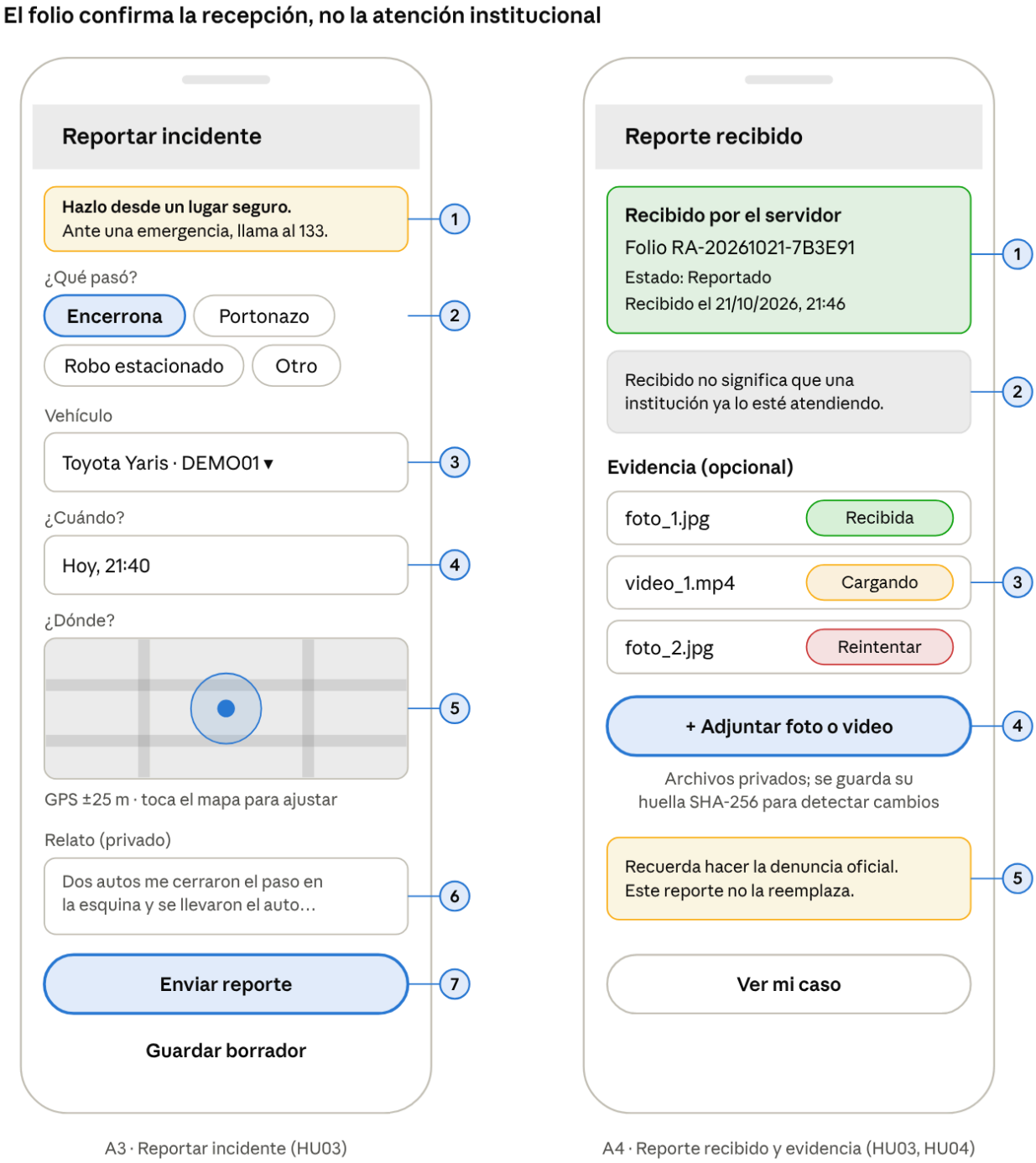


mockup A1 y A2 · app móvil, cuenta y vehículos

Marca	Elemento	Qué hace y por qué
A1 · 1	Pestañas Ingresar y Crear cuenta	Registro e inicio en un mismo lugar. Una cuenta nueva queda pendiente de verificación
A1 · 2	Correo y contraseña	El correo se normaliza: mayúsculas o espacios no crean otra cuenta. La contraseña pide al menos 8 caracteres y se guarda con scrypt
A1 · 3	Botón Ingresar	La API entrega un token de sesión de 12 horas y la base guarda sólo su huella. Si falla, dice «Credenciales inválidas» sin decir qué dato está mal
A1 · 4	¿Olvidaste tu contraseña?	Pide un token de un solo uso que vence en 30 minutos
A1 · 5	Nota de rol	El registro público sólo da rol ciudadano, nunca rol de operador

Marca	Elemento	Qué hace y por qué
A2 · 1	Tarjeta de vehículo	Marca, modelo, color y patente. El vehículo existe aparte de sus usuarios
A2 · 2	Estado del vínculo	Pendiente, autorizado, rechazado o revocado. Sólo con un vínculo pendiente o autorizado se puede reportar ese vehículo
A2 · 3	Registrar vehículo	Marca y modelo salen de un catálogo y las fotos son privadas. Si la patente ya existe, se reutiliza el vehículo y se crea una solicitud de vínculo
A2 · 4	Nota	Conocer la patente no da acceso a los datos del titular

A.2 App móvil: reportar el incidente

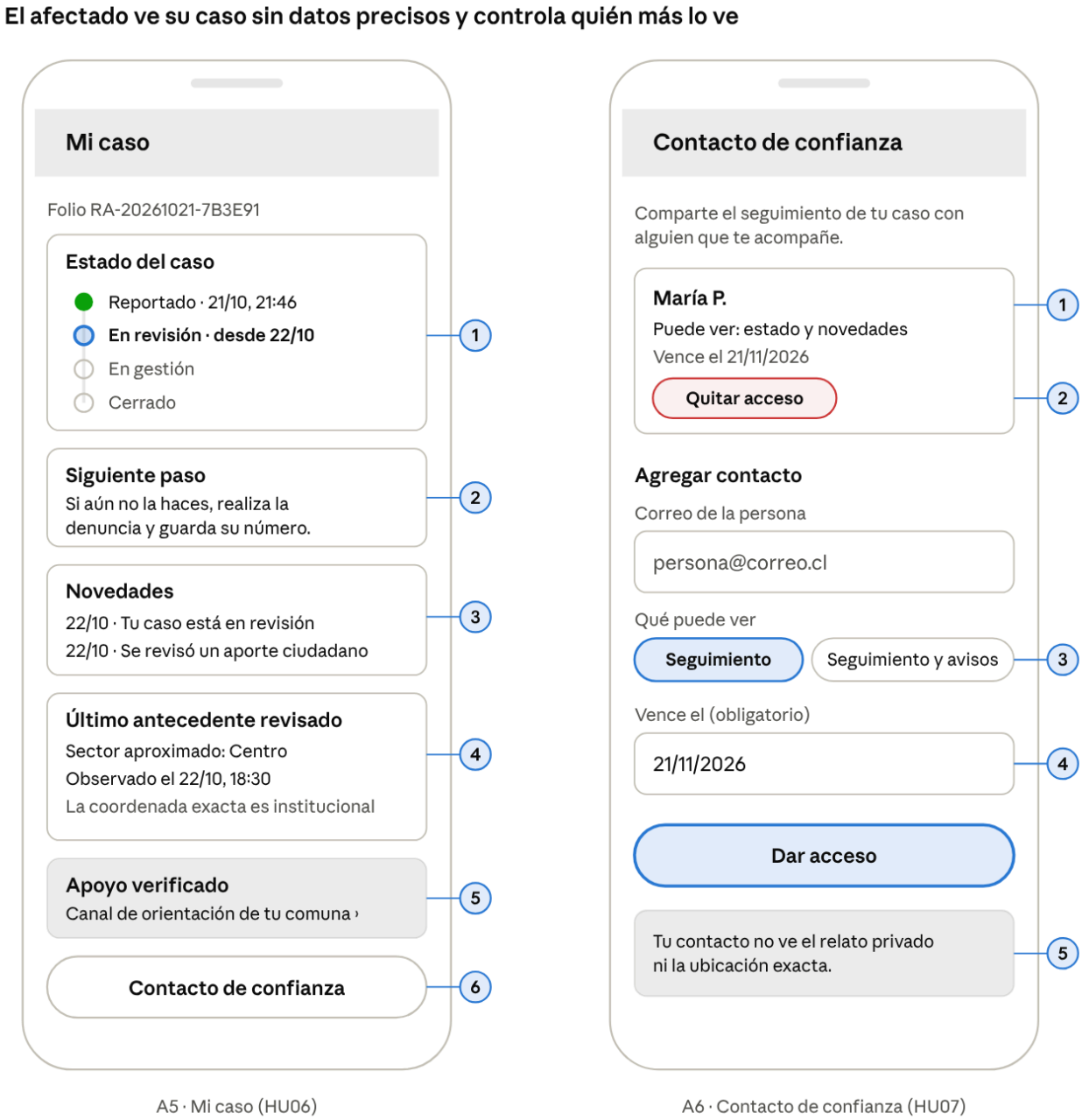


mockup A3 y A4 · app móvil, reporte y evidencia

Marca	Elemento	Qué hace y por qué
A3 · 1	Aviso de seguridad	Recuerda reportar desde un lugar seguro. Una persona que conduce no debe llenar formularios

Marca	Elemento	Qué hace y por qué
A3 · 2	Tipo de incidente	Encerrona, portonazo, robo estacionado u otro. Si no se elige, queda sin clasificar. Alimenta el indicador M05
A3 · 3	Vehículo	Sólo aparecen vehículos con vínculo pendiente o autorizado; si no, la API responde 403
A3 · 4	Cuándo	Fecha y hora del hecho. Una fecha futura se rechaza con 422
A3 · 5	Dónde	El GPS propone el punto y la persona lo corrige. Se guarda como punto PostGIS con su precisión en metros
A3 · 6	Relato privado	Sólo lo ve la institución; nunca se publica
A3 · 7	Enviar o guardar borrador	El envío lleva una Idempotency-Key: si la red falla, el reintento devuelve el mismo folio. Sin conexión queda como borrador local y no se presenta como atendido
A4 · 1	Folio y estado	Respuesta del servidor (201) con folio, estado y hora de recepción
A4 · 2	Aclaración	Recibido no es atendido. El caso puede quedar sin organización receptora
A4 · 3	Lista de evidencia	Estados pendiente, cargando, recibido o error. Sólo se marca recibido con huella, tamaño y clave del archivo
A4 · 4	Adjuntar	Fotos y videos privados en almacenamiento de objetos. La huella detecta cambios, pero no prueba que la escena sea verdadera
A4 · 5	Recordatorio de denuncia	El caso no reemplaza la denuncia oficial; si se hace, se guarda su referencia

A.3 App móvil: seguimiento del afectado

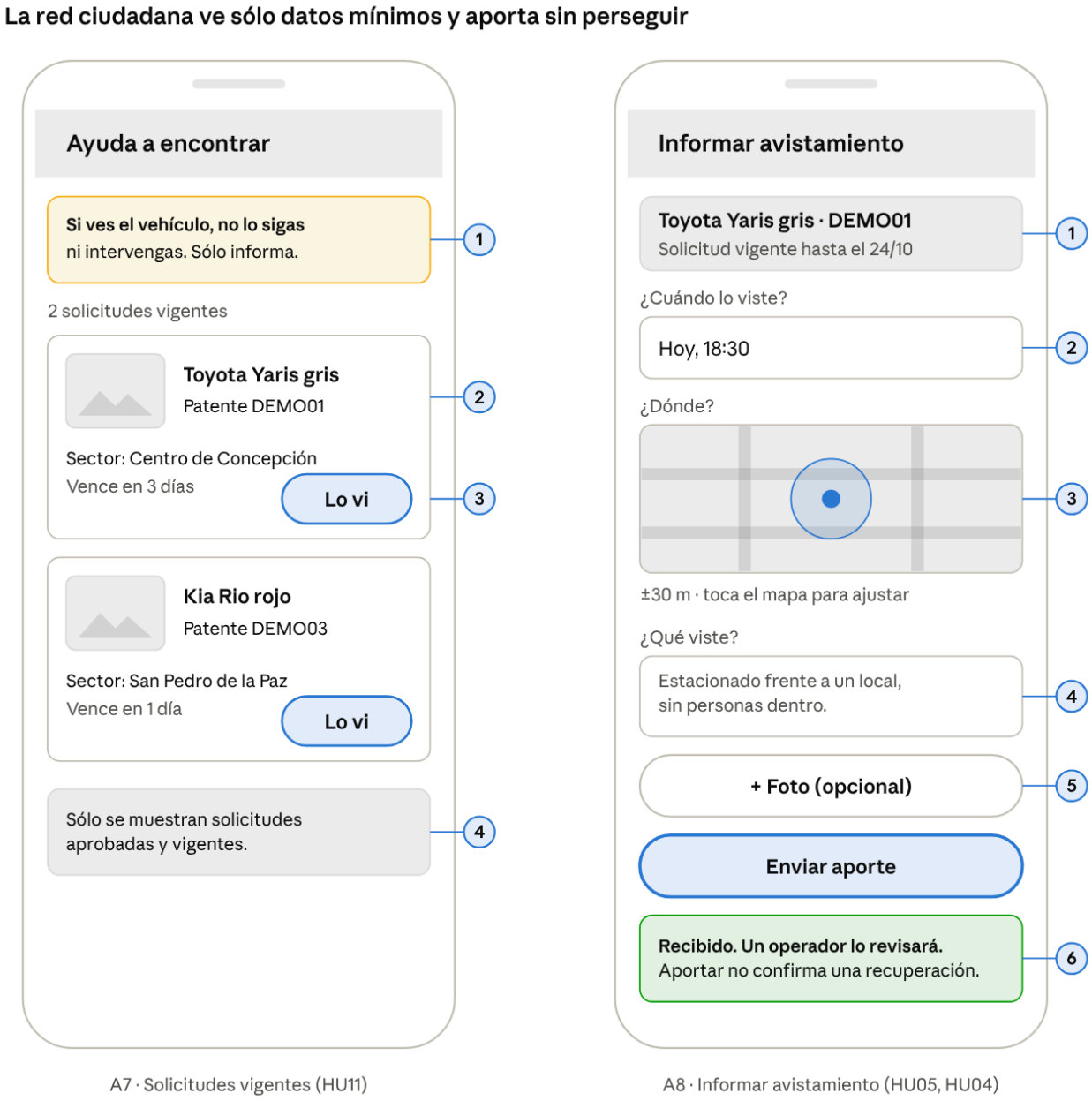


mockup A5 y A6 · app móvil, seguimiento y contacto de confianza

Marca	Elemento	Qué hace y por qué
A5 · 1	Estado del caso	Los cuatro estados con su fecha. Sale de la vista `v_seguimiento_limitado`, que no trae relato ni coordenadas
A5 · 2	Siguiente paso	Orientación concreta. Si no hay novedades, no se inventan avances
A5 · 3	Novedades	Sólo las gestiones marcadas como visibles para el afectado; las internas no aparecen

Marca	Elemento	Qué hace y por qué
A5 · 4	Último antecedente	Sector aproximado y hora. La coordenada exacta queda en el ámbito institucional. Un aporte revisado no es una recuperación
A5 · 5	Apoyo verificado	Canal vigente del catálogo de recursos de apoyo (HU22). Orienta; no diagnostica
A5 · 6	Contacto de confianza	Lleva a la pantalla A6
A6 · 1	Contacto vigente	Muestra qué puede ver y hasta cuándo
A6 · 2	Quitar acceso	La revocación es inmediata: la siguiente consulta del contacto se rechaza
A6 · 3	Qué puede ver	Alcance «seguimiento» o «seguimiento y avisos»
A6 · 4	Vencimiento	Obligatorio y posterior a la fecha de hoy. Nadie puede darse acceso a sí mismo
A6 · 5	Nota	El contacto no ve el relato privado ni la ubicación exacta

A.4 App móvil: colaboración ciudadana



A7 · Solicitudes vigentes (HU11)

A8 · Informar avistamiento (HU05, HU04)

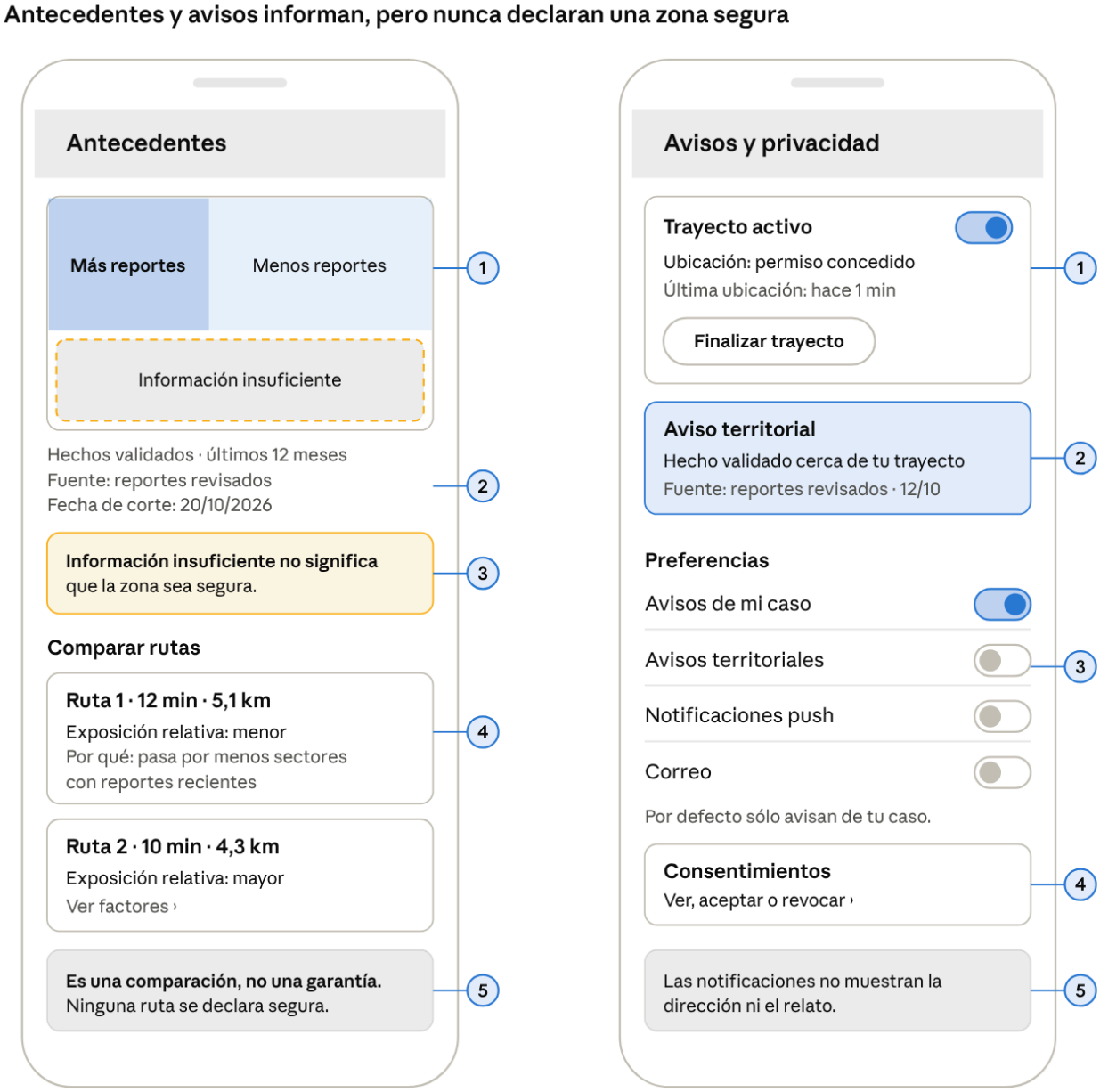
mockup A7 y A8 · app móvil, colaboración ciudadana

Marca	Elemento	Qué hace y por qué
A7 · 1	Advertencia	Un aporte es una observación, no una persecución. Nadie debe seguir el vehículo ni intervenir
A7 · 2	Tarjeta de solicitud	Sólo datos aprobados: vehículo, patente, sector publicable y vencimiento. Sale de la vista `v_solicitudes_publicables`, sin dirección, coordenadas ni relato
A7 · 3	Botón Lo vi	Abre la pantalla A8 ligada a esa solicitud

Marca	Elemento	Qué hace y por qué
A7 · 4	Sólo vigentes	Las solicitudes vencidas o retiradas desaparecen y no aceptan aportes
A8 · 1	Solicitud asociada	El aporte queda ligado al caso y a una solicitud vigente
A8 · 2	Cuándo	Hora de la observación. No puede ser futura ni posterior a la recepción
A8 · 3	Dónde	Punto con su precisión en metros. El afectado verá sólo el sector
A8 · 4	Qué viste	Descripción breve que revisará el operador
A8 · 5	Foto opcional	Se guarda como evidencia privada con su huella
A8 · 6	Confirmación	El aporte queda pendiente de revisión. Un reenvío no lo duplica, gracias a la clave de envío. Aportar no confirma una recuperación

A.5 App móvil: antecedentes, rutas y avisos

Estas pantallas son Should y Could: su alcance se acuerda con los docentes. Los números de las rutas son de ejemplo.



mockup A9 y A10 · app móvil, antecedentes, rutas y avisos

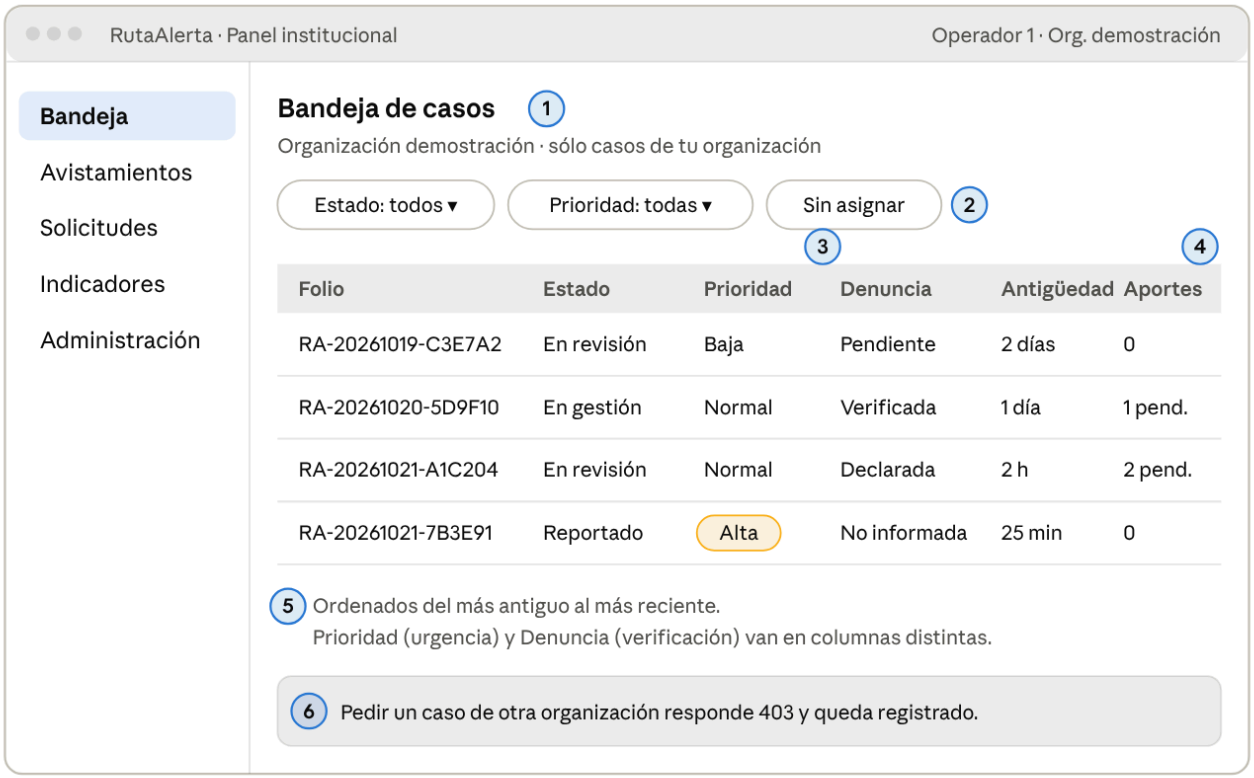
Marca	Elemento	Qué hace y por qué
A9 · 1	Mapa por sectores	Muestra hechos consolidados agrupados por sector (M06), nunca puntos privados
A9 · 2	Procedencia y corte	Cada dato dice de dónde viene, qué período cubre y su fecha de corte
A9 · 3	Información insuficiente	Se declara en vez de pintar la zona como segura
A9 · 4	Comparar rutas	Distancia, tiempo y exposición relativa, con sus factores explicados. El puntaje es relativo, no una probabilidad de delito

Marca	Elemento	Qué hace y por qué
A9 · 5	Advertencia	Es una comparación, no una garantía. Si faltan datos, el sistema se abstiene de calificar
A10 · 1	Trayecto activo	Muestra el estado del permiso y la antigüedad de la ubicación. Si el permiso se niega, advierte el límite. No sigue a ningún otro vehículo
A10 · 2	Aviso territorial	Llega con fecha y fuente. Se registra por separado cuándo se generó y cuándo se mostró
A10 · 3	Preferencias	Por defecto sólo están activos los avisos del caso; territoriales, push y correo parten apagados
A10 · 4	Consentimientos	Se registran por finalidad y versión del texto. Revocar deja fecha
A10 · 5	Contenido mínimo	Las notificaciones no llevan dirección exacta ni relato privado

A.6 Panel web: bandeja de casos

El panel tiene un menú lateral fijo: Bandeja, Avistamientos, Solicitudes, Indicadores y Administración. Cada persona ve sólo las opciones de su rol.

La bandeja muestra sólo casos propios y separa urgencia de verificación



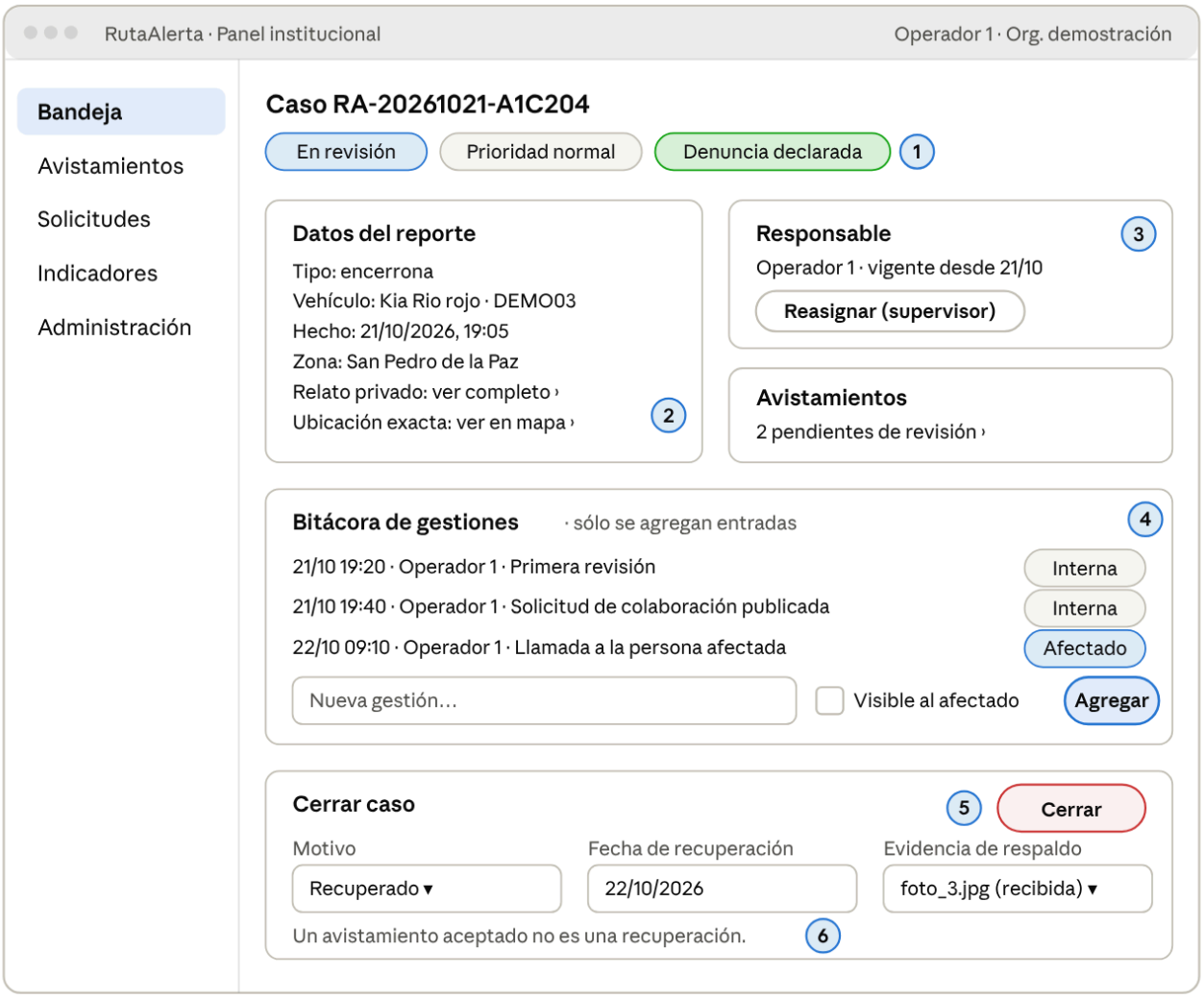
P1 · Bandeja de casos (HU13)

mockup P1 · panel web, bandeja de casos

Marca	Elemento	Qué hace y por qué
P1 · 1	Encabezado y ámbito	Sólo aparecen casos de las organizaciones donde el operador tiene membresía activa. La API lee la vista `v_bandeja_institucional`
P1 · 2	Filtros	Estado y prioridad ya existen en `GET /casos`. «Sin asignar» es una propuesta
P1 · 3	Prioridad y Denuncia	Prioridad es urgencia; Denuncia indica cuánto está verificado (no informada, pendiente, declarada o verificada). Un caso puede ser urgente y no estar verificado
P1 · 4	Aportes	Avistamientos pendientes de revisión. Ayuda a medir el tiempo de primera revisión (M09)
P1 · 5	Orden	Del más antiguo al más reciente, como lo hace hoy la API
P1 · 6	Acceso ajeno	Un caso de otra organización responde 403 y queda registrado (CP44)

A.7 Panel web: detalle del caso

El caso deja historia: cada gestión se agrega y el cierre exige respaldo



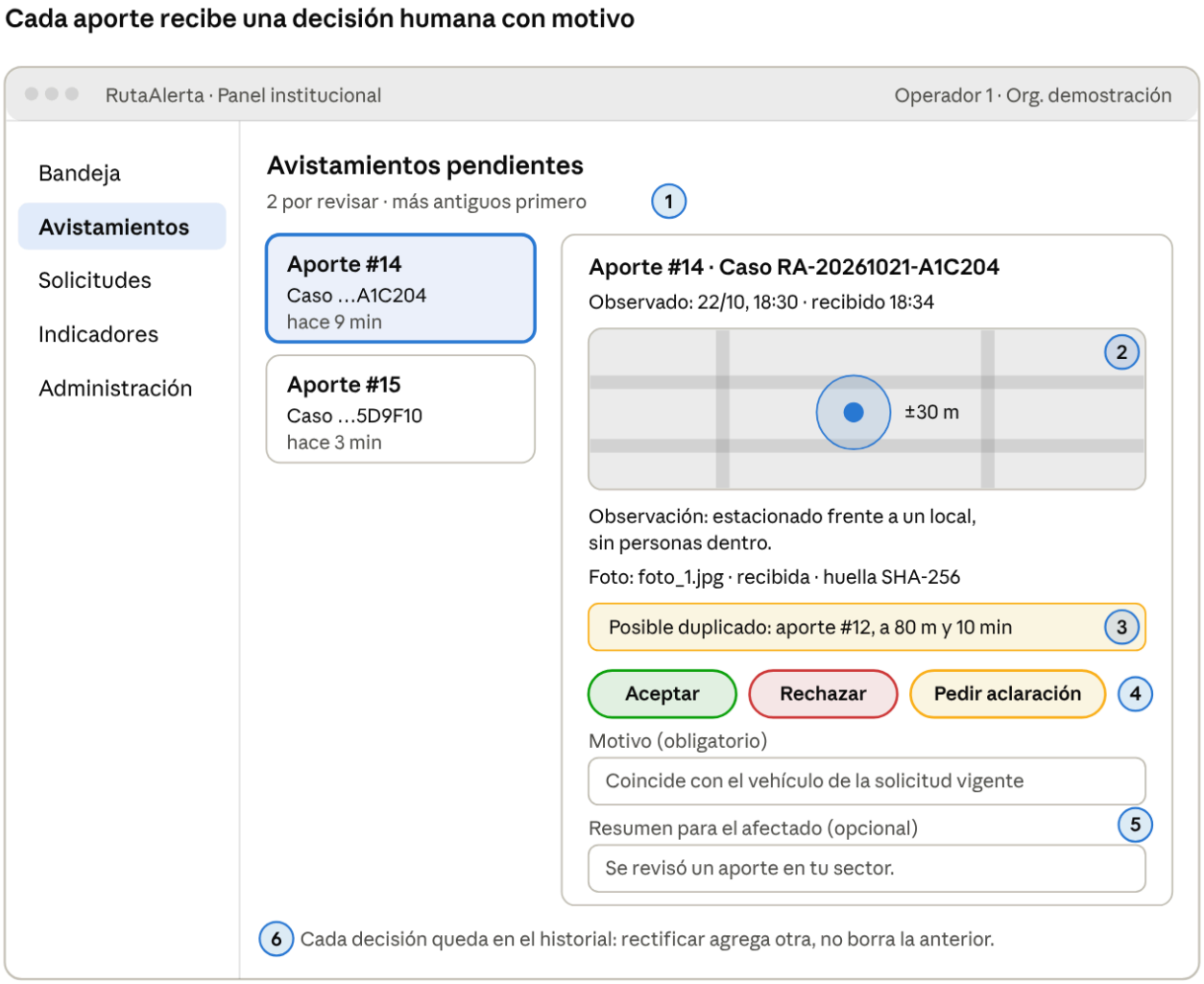
P2 · Detalle del caso (HU15, HU16)

mockup P2 · panel web, detalle del caso

Marca	Elemento	Qué hace y por qué
P2 · 1	Encabezado	Estado, prioridad y denuncia por separado
P2 · 2	Datos del reporte	El operador sí ve el relato privado y la ubicación exacta, porque está en el ámbito institucional
P2 · 3	Responsable	Una sola asignación vigente por caso; reasignar cierra la anterior. Sólo un supervisor de la misma organización puede hacerlo
P2 · 4	Bitácora	Cada gestión se agrega y nunca se edita. «Visible al afectado» la muestra en la pantalla A5

Marca	Elemento	Qué hace y por qué
P2 · 5	Cerrar caso	Exige motivo y fecha. Con motivo «recuperado» exige además la fecha de recuperación y una evidencia del mismo caso
P2 · 6	Nota	Un avistamiento aceptado no permite cerrar como recuperado

A.8 Panel web: revisión de avistamientos



P3 · Revisión de avistamientos (HU14)

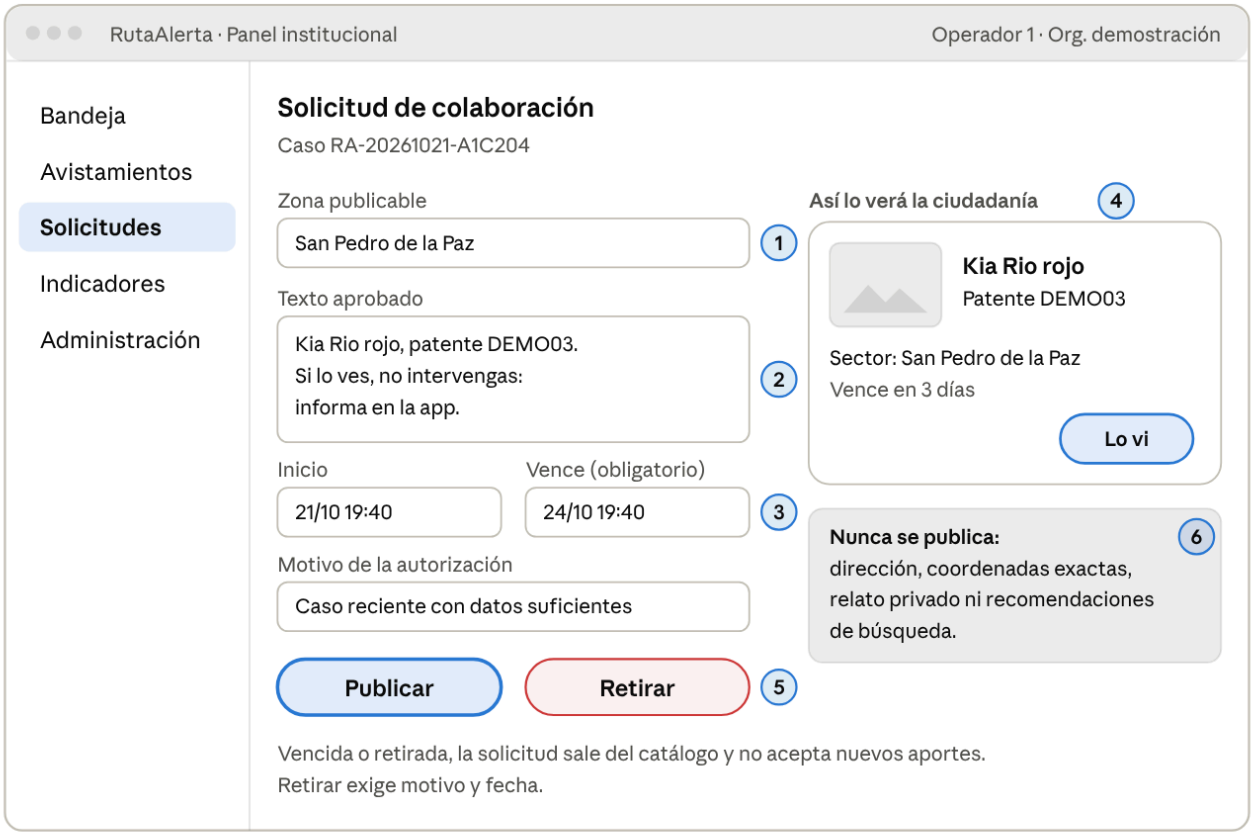
mockup P3 · panel web, revisión de avistamientos

Marca	Elemento	Qué hace y por qué
P3 · 1	Cola de pendientes	Los más antiguos primero. El tiempo hasta la primera decisión alimenta M09 (vista `v_revisiones_oportunas`)
P3 · 2	Datos del aporte	Hora observada y de recepción, punto con precisión, observación y foto con huella. El operador sí ve la ubicación exacta

Marca	Elemento	Qué hace y por qué
P3 · 3	Posible duplicado	Avisa de aportes parecidos para que la persona decida. Es una propuesta: no fusiona nada sola
P3 · 4	Decisión y motivo	Aceptar, rechazar o pedir aclaración, siempre con motivo. La función `revisar_avistamiento` guarda decisión, estado, evento y auditoría en una transacción. Sólo decide un miembro activo de la organización del caso
P3 · 5	Resumen para el afectado	Texto breve y cuidado que la persona verá en la pantalla A5
P3 · 6	Historial	Las decisiones no se editan. Rectificar agrega una nueva

A.9 Panel web: solicitud de colaboración

Se publica un texto aprobado, con vencimiento y sin datos privados



P4 · Solicitud de colaboración (HU17)

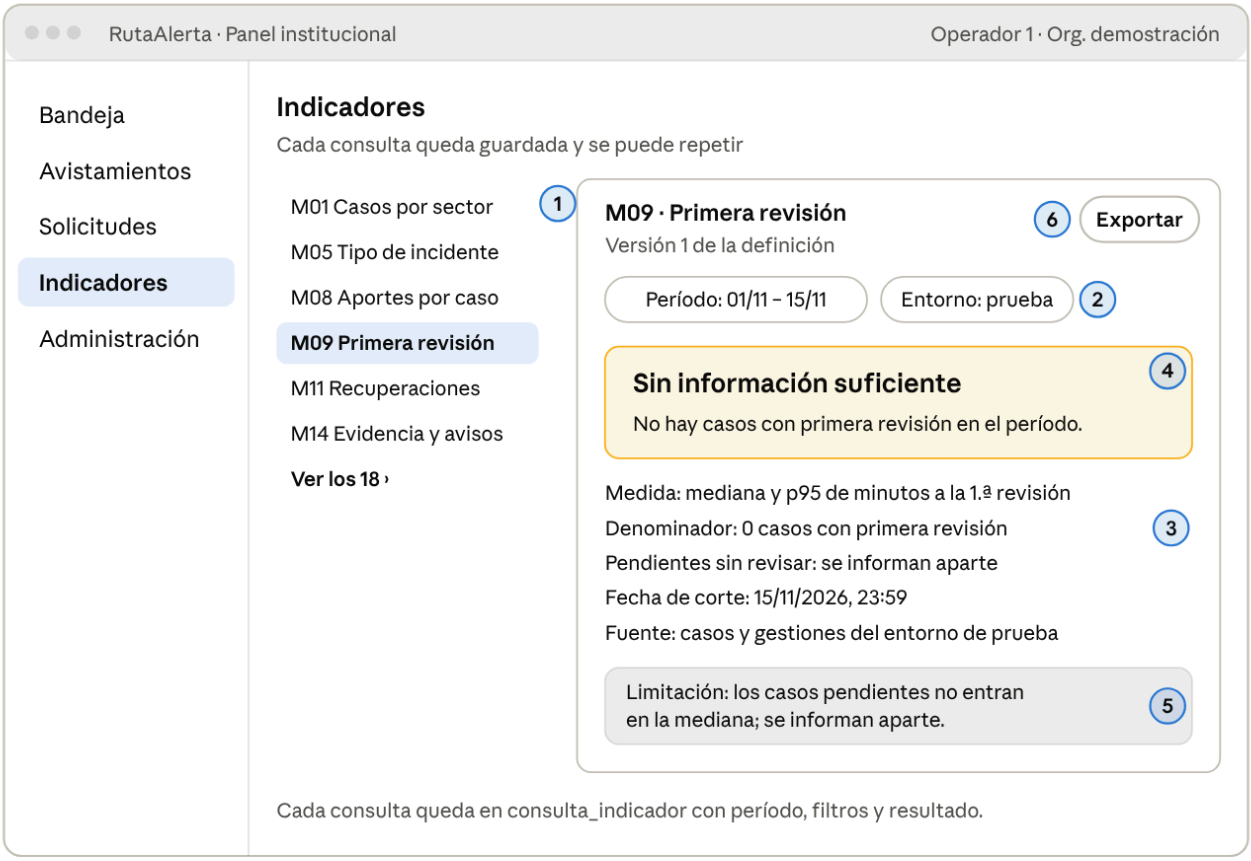
mockup P4 · panel web, solicitud de colaboración

Marca	Elemento	Qué hace y por qué
P4 · 1	Zona publicable	Un sector general, nunca la dirección

Marca	Elemento	Qué hace y por qué
P4 · 2	Texto aprobado	Redactado y revisado por el operador. Incluye la advertencia de no intervenir
P4 · 3	Vigencia	Inicio y vencimiento obligatorios; el vencimiento va después del inicio
P4 · 4	Vista previa	Muestra exactamente lo que verá la red ciudadana en la pantalla A7
P4 · 5	Publicar o retirar	Retirar exige motivo y fecha. Una solicitud retirada o vencida no acepta aportes
P4 · 6	Nunca se publica	Dirección, coordenadas exactas, relato privado ni recomendaciones de búsqueda

A.10 Panel web: indicadores

Un indicador sin datos lo dice, en vez de mostrar un cero



P5 · Indicadores (HU18)

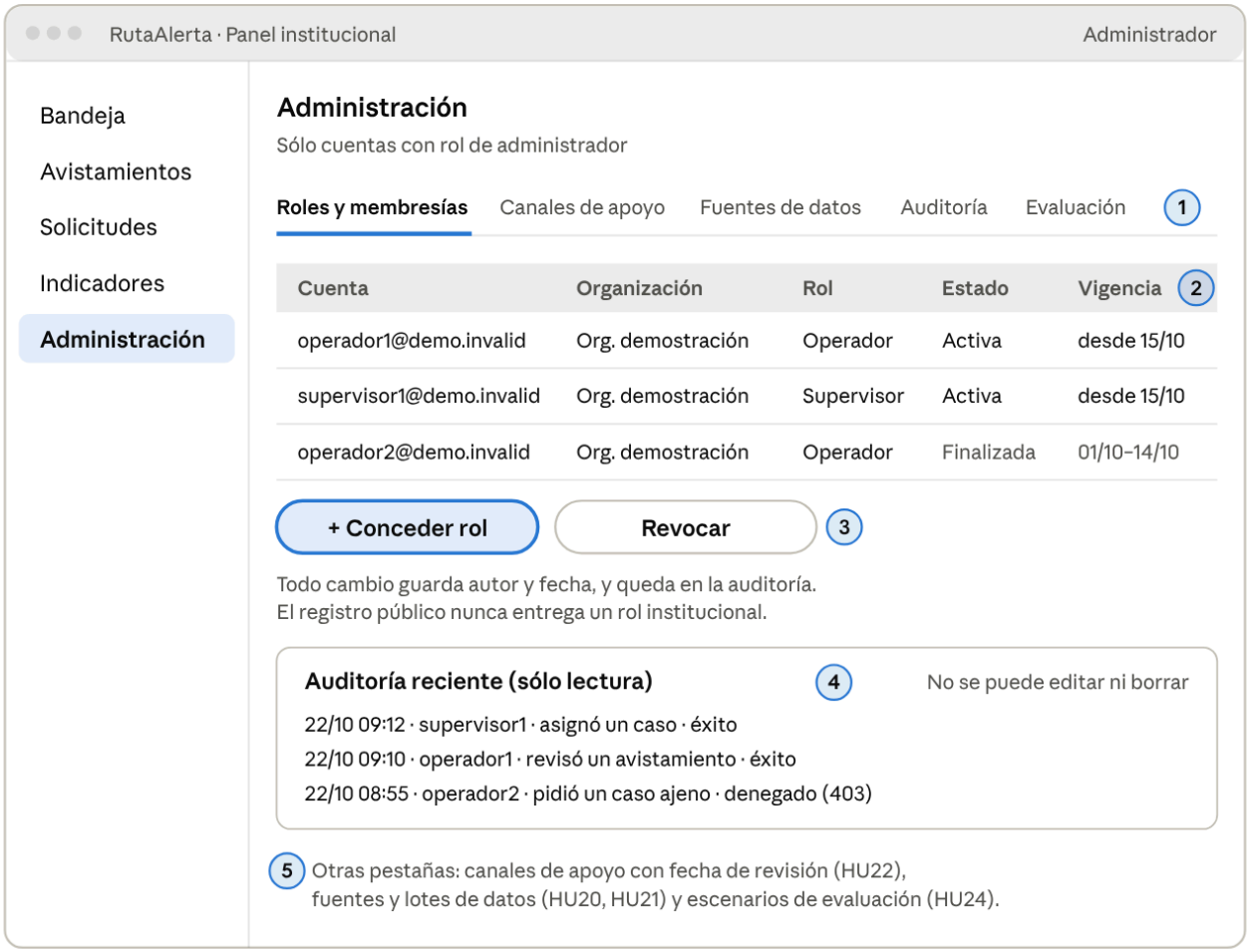
mockup P5 · panel web, indicadores

Marca	Elemento	Qué hace y por qué
P5 · 1	Catálogo	Los 18 indicadores, cada uno con su definición versionada
P5 · 2	Filtros	Período y entorno (prueba u operación), para no mezclar datos de ensayo con datos reales

Marca	Elemento	Qué hace y por qué
P5 · 3	Ficha del cálculo	Medida, denominador, pendientes aparte, fecha de corte y fuente: todo lo necesario para repetir el cálculo
P5 · 4	Sin información suficiente	Si el denominador es cero, no se muestra un 0 ficticio. El estado queda `sin_datos`
P5 · 5	Limitación	Cada indicador declara lo que no mide
P5 · 6	Exportar	La consulta se guarda en `consulta_indicador` con período, filtros y resultado

A.11 Panel web: administración

Los permisos se conceden con autor y fecha, y la auditoría no se edita



P6 · Administración (HU19 a HU24)

mockup P6 · panel web, administración

Marca	Elemento	Qué hace y por qué
P6 · 1	Pestañas	Roles y membresías (HU19), canales de apoyo (HU22), fuentes de datos (HU20 y HU21), auditoría (HU23) y evaluación (HU24)
P6 · 2	Membresías	Rol por organización, con estado y vigencia. Una persona tiene a lo más una membresía vigente por organización
P6 · 3	Conceder o revocar	Guarda autor y fecha y queda auditado. El registro público nunca entrega un rol institucional
P6 · 4	Auditoría	Sólo lectura, incluidos los accesos denegados. Un trigger impide editarla o borrarla
P6 · 5	Otras pestañas	Canales de apoyo con territorio y fecha de revisión; fuentes y lotes que entran pendientes de revisión; escenarios de prueba versionados

A.12 Panel web: búsqueda asistida (propuesta)

Esta pantalla corresponde a OE5, que los docentes no han ratificado. Depende de un convenio o de una cámara propia y de modelos de IA. Se incluye para explicar el diseño, no como compromiso.

Propuesta: la IA ordena qué revisar, pero la decisión siempre es humana



P7 · Búsqueda asistida (HU26 a HU31) · propuesta

mockup P7 · panel web, búsqueda asistida (propuesta)

Marca	Elemento	Qué hace y por qué
P7 · 1	Aviso de propuesta	Deja claro que es una ampliación no comprometida y que los datos son simulados
P7 · 2	Mapa con corredores	Hipótesis versionadas desde una observación ancla, con tiempos y supuestos. Nunca una posición confirmada, y nunca a partir del GPS de la víctima
P7 · 3	Cámaras recomendadas	Prioridad explicada con su intervalo. El operador acepta, descarta o deja indeterminado, con motivo
P7 · 4	Candidatos visuales	Ordenados por semejanza, con la versión del modelo. La puntuación no confirma identidad
P7 · 5	Copiloto	Separa hechos, hipótesis y limitaciones, con referencias verificables. Si falta información, lo dice
P7 · 6	Decisión humana	La IA no publica ni cierra casos. Ver video exige registrar caso, cámara, operador e intervalo

A.13 Principios de diseño que se repiten

Principio	Cómo se ve en las pantallas	Dónde
Datos mínimos	La red ciudadana y el afectado ven un sector aproximado; la coordenada exacta es institucional	A5, A7, P2, P4
Estados honestos	Recibido no es atendido, aceptado no es recuperado y sin datos no es cero	A4, A8, P2, P5
Decisión humana con motivo	Revisar, cerrar y recomendar exigen motivo y dejan historial	P2, P3, P7
Control de la persona	Contacto de confianza revocable, preferencias y consentimientos	A6, A10
Seguridad primero	Reportar desde un lugar seguro y no seguir ni intervenir	A3, A7
Conceptos separados	Urgencia frente a verificación; hecho frente a hipótesis	P1, P7
Sin promesas	Ninguna ruta se declara segura ni se predice un delito	A9

Relación con las heurísticas de Nielsen

El DAS (sección 8) evalúa la usabilidad con las 10 heurísticas de Nielsen. Estos son ejemplos de cómo los mockups las aplican:

Heurística	Ejemplo en los mockups
Visibilidad del estado del sistema	Estados de carga de la evidencia (A4) y estado del caso paso a paso (A5)
Coincidencia con el mundo real	Palabras de uso diario: encerrona, portonazo, «Lo vi» (A3, A7)
Control y libertad del usuario	Guardar borrador (A3), quitar acceso (A6), retirar una solicitud (P4)
Consistencia y estándares	Los mismos colores para los mismos estados en la app y el panel
Prevención de errores	Sólo vehículos vinculados (A3); el vencimiento es obligatorio (A6, P4)
Reconocer antes que recordar	Botones para el tipo de incidente (A3); marcas y modelos de un catálogo (A2)
Flexibilidad y eficiencia	Filtros de la bandeja (P1)
Diseño estético y minimalista	Cada pantalla tiene una sola acción principal
Ayudar a recuperarse de errores	«Reintentar» en una carga fallida (A4)
Ayuda y documentación	Notas breves que explican cada regla dentro de la pantalla

Cómo seguir

1. Revisar estos mockups con el equipo y con los docentes.
2. Pasar las pantallas aprobadas a un prototipo navegable, por ejemplo en Figma.
3. Probar el prototipo con usuarios antes de programar y guardar resultados reales, con fecha y participantes.

4. Actualizar la sección 7 del DAS (Prototipo y mockups) con la versión aprobada.